Complete SDE Intern Interview Cheatsheet

Educational Guide: From Basics to Your Project

Aditya Shubham | Amazon ATLAS Program | Online Bookstore Application

HOW TO USE THIS CHEATSHEET

Structure: Each concept follows this pattern:

1. **Definition:** What is it?
2. **Why Use It:** Purpose and benefits
3. **General Example:** Simple, universal example
4. **Your Project:** How you used it in bookstore
5. **Interview Answer:** Ready-to-use response

Color Coding:

- Blue boxes = Definitions
- Yellow boxes = General examples
- Green boxes = Your project
- Orange boxes = Interview answers

PROJECT ELEVATOR PITCH

Interview Answer Template

"Tell me about your project" (30 seconds):

"I built a full-stack online bookstore using Java Spring Boot and React. The application features 20+ REST APIs, DynamoDB for cloud storage, and personalized book recommendations. I implemented an Amazon-style ASIN system for inventory management that prevents duplicate entries. The project has 134 automated tests with 58% coverage and a CI/CD pipeline using GitHub Actions. It demonstrates all core OOP principles and SOLID design patterns."

Quick Stats (Memorize):

- **Duration:** 10 days
- **Code:** 6,300+ lines (3,500 backend + 2,800 frontend)
- **Tests:** 134 tests, 58% coverage
- **Features:** Search, Cart, Checkout, History, Recommendations, Admin Panel
- **Tech:** Java 17, Spring Boot, React 18, DynamoDB, Material-UI

OBJECT-ORIENTED PROGRAMMING

1. ENCAPSULATION

Definition

What is Encapsulation?

Encapsulation means bundling data (variables) and methods (functions) that operate on that data into a single unit (class), and restricting direct access to some of the object's components. This is achieved using access modifiers: `private`, `protected`, `public`.

Why Use It?

Benefits:

- **Data Protection:** Prevents accidental modification
- **Flexibility:** Change implementation without breaking code
- **Maintainability:** Clear interface vs internal logic
- **Validation:** Control how data is accessed/modified

General Example

Simple Example: Bank Account

```java
public class BankAccount {
  // Private - hidden from outside
  private double balance;

  // Public - exposed interface
  public void deposit(double amount) {
    if (amount > 0) {  // Validation
      balance += amount;
    }
  }

  public double getBalance() {
    return balance;  // Controlled access
  }
}

// Usage
BankAccount acc = new BankAccount();
acc.deposit(100);  // OK
acc.balance = 1000000;  // ERROR! Can't access private
```

Why this matters:

- Can't set negative balance
- Can't bypass deposit rules
- Can change how balance is stored internally

In Your Bookstore Project

In Your Bookstore - Cart Class:

```java
public class Cart {
  // PRIVATE - hidden from controllers/users
  private final Map<String, CartItem> items
    = new LinkedHashMap<>();

  // PUBLIC - exposed methods
  public synchronized void addBook(
      Book book, int quantity) {
    // Internal validation
    if (quantity <= 0) {
      throw new IllegalArgumentException(
        "Quantity must be positive");
    }

    String key = book.getId();
    if (items.containsKey(key)) {
      CartItem existing = items.get(key);
      existing.incrementQuantity(quantity);
    } else {
      items.put(key,
        new CartItem(book, quantity));
    }
  }

  public BigDecimal calculateTotal() {
    // Users don't see how total is calculated
    return items.values().stream()
      .map(CartItem::totalPrice)
      .reduce(BigDecimal.ZERO,
              BigDecimal::add);
  }

  public Map<String, CartItem> getItems() {
    // Return unmodifiable copy
    return Collections.unmodifiableMap(items);
  }
}
```

What you encapsulated:

1. **Data:** `items` map is private
2. **Logic:** How cart calculates total is hidden
3. **Validation:** Quantity checks inside method
4. **Thread-safety:** Synchronized keyword

Benefits you gained:

- Changed HashMap LinkedHashMap without breaking controllers
- Added quantity validation in one place
- Prevented concurrent modification issues
- Can log cart operations without changing API

Interview Answer Template

Interview Answer:
"Encapsulation means hiding internal implementation details and exposing only necessary interfaces. In my bookstore, the Cart class uses a private LinkedHashMap to store items, but only exposes public synchronized methods like addBook() and calculateTotal(). This protects the cart data from invalid states - for example, users can't add negative quantities or access the map directly. When I changed from HashMap to LinkedHashMap to preserve insertion order, I only had to modify the Cart class - no controller code broke because they only used the public interface."
Follow-up: "Why synchronized?"
"Because multiple HTTP requests might access the same cart simultaneously in a multi-threaded web server like Tomcat. Without synchronization, two users clicking 'add to cart' at the same time could cause data corruption. The synchronized keyword ensures only one thread modifies the cart at a time."

2. INHERITANCE

Definition

What is Inheritance?
Inheritance is when a class (child/subclass) acquires properties and behaviors from another class (parent/superclass). The child class can use parent's methods, override them, or add new ones. Uses `extends` keyword.

Why Use It?

Benefits:
- **Code Reuse:** Don't repeat common functionality
- **Hierarchy:** Organize related classes
- **Polymorphism:** Treat children as parent type
- **Extensibility:** Add features without modifying parent

General Example

Simple Example: Shapes

```java
// Parent class
public class Shape {
  protected String color;

  public void setColor(String color) {
    this.color = color;
  }

  public double area() {
    return 0;  // Default implementation
  }
}

// Child class inherits from Shape
public class Circle extends Shape {
  private double radius;

  public Circle(double radius) {
    this.radius = radius;
  }

  @Override  // Override parent method
  public double area() {
    return Math.PI * radius * radius;
  }

  // Circle has setColor() automatically!
}

// Usage
Circle c = new Circle(5);
c.setColor("red");  // Inherited from Shape
double a = c.area();  // Overridden in Circle
```

In Your Bookstore Project

In Your Bookstore - Exception Hierarchy:

```java
// Parent: Java's built-in RuntimeException
public class BookNotFoundException
    extends RuntimeException {

  public BookNotFoundException(String message) {
    super(message);  // Call parent constructor
  }

  public BookNotFoundException(
      String message, Throwable cause) {
    super(message, cause);
  }
}

// Usage in BookService
public Book getBookById(String id) {
  Book book = bookTable.getItem(key);
  if (book == null) {
    throw new BookNotFoundException(
      "Book not found with id: " + id);
  }
  return book;
}

// Spring catches this automatically!
@ExceptionHandler(BookNotFoundException.class)
public ResponseEntity<?> handleNotFound(
    BookNotFoundException ex) {
  return ResponseEntity.status(404)
    .body(Map.of("error", ex.getMessage()));
}
```

What you inherited:
1. **From RuntimeException:**
   - `getMessage()` method
   - `printStackTrace()` method
   - Stack trace functionality
   - Exception handling mechanism
2. **From Spring @RestController:**
   - HTTP request/response handling
   - JSON serialization
   - Exception handling

Repository Interface Pattern:

```java
// Interface (like a contract)
public interface UserRepository {
  User findByUsername(String username);
  void save(User user);
  void delete(String username);
}

// Implementation 1: DynamoDB
public class DynamoDbUserRepository
    implements UserRepository {

  @Override
  public User findByUsername(String username) {
    // DynamoDB-specific code
    return userTable.getItem(key);
  }

  @Override
  public void save(User user) {
    userTable.putItem(user);
  }
}

// Could have Implementation 2: MySQL
public class MySQLUserRepository
    implements UserRepository {

  @Override
  public User findByUsername(String username) {
    // SQL query
    return jdbcTemplate.queryForObject(sql, ...);
  }
}

// Service doesn't care which implementation!
public class UserService {
  private final UserRepository repository;
```

Interview Answer Template

Interview Answer:
"Inheritance lets a class acquire properties and behaviors from another class. In my bookstore, I created BookNotFoundException that extends RuntimeException. This means my custom exception automatically gets all the functionality of Java's exception system - stack traces, getMessage(), error handling - without writing any of that code myself. I just added bookstore-specific logic."
Another example:
"I used interface inheritance with the Repository pattern. UserRepository is an interface with methods like findByUsername() and save(). I created DynamoDbUserRepository that implements this interface. The benefit is I could create a MySQLUserRepository later without changing any service code - they both follow the same contract. This is also polymorphism."

3. POLYMORPHISM

Definition

What is Polymorphism?
Polymorphism means "many forms". It allows objects of different classes to be treated as objects of a common parent class. The same method call behaves differently based on the object's actual type. Two types:

- **Compile-time:** Method overloading (same name, different parameters)
- **Runtime:** Method overriding (interface/inheritance)

Why Use It?

Benefits:

- **Flexibility:** One interface, multiple implementations
- **Extensibility:** Add new types without changing existing code
- **Testability:** Easy to mock/stub dependencies
- **Maintainability:** Change implementation without changing callers

General Example

Simple Example: Animal Sounds

```
// Parent class
abstract class Animal {
  abstract void makeSound();
}

// Child classes
class Dog extends Animal {
  @Override
  void makeSound() {
    System.out.println("Woof!");
  }
}

class Cat extends Animal {
  @Override
  void makeSound() {
    System.out.println("Meow!");
  }
}

// Polymorphism in action
public static void main(String[] args) {
  // All are Animal type, but different objects
  Animal a1 = new Dog();
  Animal a2 = new Cat();

  // Same method call, different behavior!
  a1.makeSound();  // Prints "Woof!"
  a2.makeSound();  // Prints "Meow!"

  // Can store in same array
  Animal[] animals = {new Dog(), new Cat(), new Dog()};
  for (Animal animal : animals) {
    animal.makeSound();  // Calls correct version
  }
}
```

Key Point: The variable type is `Animal`, but the actual object type determines which `makeSound()` runs.

In Your Bookstore Project

In Your Bookstore - Repository Pattern:

```
// Interface (contract)
public interface OrderRepository {
  void save(Order order);
  Optional<Order> findById(String orderId);
  List<Order> findByUsername(String username);
}

// Implementation 1: In-Memory (for development)
public class InMemoryOrderRepository
    implements OrderRepository {

  private final Map<String, Order> orders
    = new HashMap<>();

  @Override
  public void save(Order order) {
    orders.put(order.getOrderId(), order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    return Optional.ofNullable(orders.get(orderId));
  }
}

// Implementation 2: DynamoDB (for production)
public class DynamoDbOrderRepository
    implements OrderRepository {

  private final DynamoDbTable<Order> orderTable;

  @Override
  public void save(Order order) {
    orderTable.putItem(order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    Order order = orderTable.getItem(key);
    return Optional.ofNullable(order);
  }
}

// Service uses interface (polymorphism!)
@Service
public class CheckoutService {
  // Can be ANY implementation
  private final OrderRepository orderRepository;

  public CheckoutService(
      OrderRepository orderRepository) {
    this.orderRepository = orderRepository;
  }

  public Order checkout(Cart cart, String username) {
    // Create order
    Order order = new Order(
      UUID.randomUUID().toString(),
      username,
      cart.getItems(),
      cart.calculateTotal()
    );

    // Save - doesn't care if in-memory or DynamoDB!
    orderRepository.save(order);

    return order;
  }
}

// Spring configuration chooses implementation
@Configuration
public class AppConfig {
  @Bean
  public OrderRepository orderRepository() {
    // Development
    return new InMemoryOrderRepository();

    // Production (just change this line!)
    // return new DynamoDbOrderRepository(client);
  }
}
```

How polymorphism helped you:

1. **Development:** Used in-memory repository (fast, no setup)
2. **Testing:** Created mock repository for unit tests
3. **Future:** Can switch to DynamoDB by changing 1 line

Interview Answer Template

Interview Answer:
"Polymorphism means one interface, many implementations. In my bookstore, OrderRepository is an interface with save() and findById() methods. I have two implementations: InMemoryOrderRepository for development and DynamoDbOrderRepository for production. The CheckoutService depends on the OrderRepository interface, not a specific implementation. This means I can swap between implementations without changing any business logic - that's polymorphism in action."
Real benefit I experienced:
"During development, I used the in-memory version because it's fast and doesn't need DynamoDB setup. For testing, I created a mock implementation that returns fake data. For production, I can switch to DynamoDB by changing one line in Spring configuration. The checkout code never changed - it just calls repository.save() and doesn't care how it's implemented."

4. ABSTRACTION

Definition

What is Abstraction?
Abstraction means hiding complex implementation details and showing only essential features. It's achieved through abstract classes and interfaces. Users interact with simple methods without knowing the complex logic behind them.

Why Use It?

Benefits:

- **Simplicity:** Easy-to-use interfaces
- **Maintainability:** Change internals without affecting users
- **Focus:** Users focus on WHAT, not HOW
- **Reduced Complexity:** Hide messy details

General Example

Simple Example: Car Driving

```
// Abstract interface
interface Car {
  void start();
  void stop();
  void accelerate();
}

// Implementation hides complexity
class ElectricCar implements Car {
  @Override
  public void start() {
    // Complex: Battery check, motor initialization,
    // power management, safety checks...
    // But user just calls start()!
    System.out.println("Car started");
  }

  @Override
  public void accelerate() {
    // Complex: Power distribution, motor control,
    // traction control, battery monitoring...
    System.out.println("Accelerating");
  }
}

// User doesn't need to know internals
Car myCar = new ElectricCar();
myCar.start();       // Simple!
myCar.accelerate();  // Don't need to know how
```

Analogy: You drive a car by pressing pedals and turning wheel. You don't need to understand engine combustion, transmission gears, or fuel injection - those details are abstracted away.

In Your Bookstore Project

In Your Bookstore - BookService Abstraction:
Without Abstraction (messy):

```
// Controller would need to know DynamoDB details
@GetMapping("/search")
public List<Book> search(@RequestParam String query) {
  // Complex DynamoDB code in controller!
  DynamoDbTable<Book> table = enhancedClient
    .table("Books", TableSchema.fromBean(Book.class));

  PageIterable<Book> pages = table.scan();
  List<Book> allBooks = pages.items().stream()
    .collect(Collectors.toList());

  String lower = query.toLowerCase();
  return allBooks.stream()
    .filter(b ->
      b.getTitle().toLowerCase().contains(lower) ||
      b.getAuthor().toLowerCase().contains(lower))
    .collect(Collectors.toList());

  // Too complex! Controller knows too much!
}
```

With Abstraction (clean):

```
// Service abstracts complexity
@Service
public class BookService {
  private final DynamoDbTable<Book> bookTable;

  // Simple interface - hides DynamoDB details
  public List<Book> searchBooks(String query) {
    // All complexity hidden here
    PageIterable<Book> pages = bookTable.scan();
    List<Book> allBooks = pages.items().stream()
      .collect(Collectors.toList());

    String lower = query.toLowerCase();
    return allBooks.stream()
      .filter(b ->
        b.getTitle().toLowerCase().contains(lower) ||
        b.getAuthor().toLowerCase().contains(lower))
      .collect(Collectors.toList());
  }

  public Book getBookById(String id) {
    // Abstracts key creation, error handling
    Key key = Key.builder()
      .partitionValue(id)
      .build();
    return bookTable.getItem(key);
  }
}

// Controller is now simple!
@RestController
public class BooksController {
  private final BookService bookService;

  @GetMapping("/search")
  public List<Book> search(@RequestParam String query) {
    // Clean! Just call service method
    return bookService.searchBooks(query);
  }

  @GetMapping("/{id}")
  public Book getById(@PathVariable String id) {
    // Don't need to know about DynamoDB Keys
    return bookService.getBookById(id);
  }
}
```

What you abstracted:

1. **DynamoDB SDK:** Complex Enhanced Client API hidden
2. **Query Logic:** Filter/search logic in service
3. **Error Handling:** Null checks, exceptions handled in service
4. **Data Transformation:** PageIterable List conversion

Benefits you gained:

- Controllers are 5 lines instead of 20
- Can change from DynamoDB to MySQL without touching

Interview Answer Template

Interview Answer:
"Abstraction means hiding complexity behind simple interfaces. In my bookstore, the BookService abstracts away all DynamoDB operations. Controllers just call simple methods like searchBooks(query) or getBookById(id). They don't need to know about DynamoDB's Enhanced Client, Key builders, or PageIterable results. All that complexity is hidden in the service layer."

Concrete example:
"For book search, the controller just calls bookService.searchBooks(query) - one line. Behind the scenes, the service scans DynamoDB, converts PageIterable to List, filters by title and author, and handles errors. But the controller doesn't need to know any of that. If I switch from DynamoDB to Elasticsearch later, I only change the BookService - controllers stay exactly the same."

SOLID PRINCIPLES

1. SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Definition

What is SRP?
A class should have only ONE reason to change. Each class should have ONE job or responsibility. If a class does multiple things, changes to one responsibility can break the others.

Why Use It?

Benefits:

- **Easier to understand:** Clear purpose
- **Easier to test:** Fewer dependencies
- **Easier to maintain:** Changes are isolated
- **Less coupling:** Classes don't depend on unrelated features

General Example

BAD Example - Multiple Responsibilities:

```
// This class does TOO MUCH!
public class User {
  private String username;
  private String email;

  // Responsibility 1: User data
  public String getUsername() { ... }

  // Responsibility 2: Database operations
  public void saveToDatabase() {
    // SQL code here
  }

  // Responsibility 3: Email sending
  public void sendWelcomeEmail() {
    // Email code here
  }

  // Responsibility 4: Validation
  public boolean isValidEmail() {
    // Regex validation
  }
}
// Problem: Changing email format affects database code!
```

GOOD Example - Single Responsibilities:

```
// Class 1: Just data (Model)
public class User {
  private String username;
  private String email;

  public String getUsername() { return username; }
  public String getEmail() { return email; }
}

// Class 2: Just database (Repository)
public class UserRepository {
  public void save(User user) {
    // SQL code
  }

  public User findByUsername(String username) {
    // SQL code
  }
}

// Class 3: Just email (Service)
public class EmailService {
  public void sendWelcomeEmail(User user) {
    // Email code
  }
}

// Class 4: Just validation (Validator)
public class UserValidator {
  public boolean isValidEmail(String email) {
    // Regex validation
  }
}

// Now changing email validation doesn't affect database!
```

In Your Bookstore Project

In Your Bookstore - Separation of Concerns:
Each class has ONE job:

```
// 1. Book.java - ONLY data (Model)
@DynamoDbBean
public class Book {
  private String id;
  private String title;
  private String author;
  private BigDecimal price;

  // Just getters/setters - no business logic
}

// 2. BookService.java - ONLY business logic
@Service
public class BookService {
  // One job: Book business operations
  public List<Book> searchBooks(String query) { ... }
  public Book getBookById(String id) { ... }
  public void saveBook(Book book) { ... }

  // Does NOT handle HTTP, does NOT validate input,
  // does NOT format responses
}

// 3. BooksController.java - ONLY HTTP handling
@RestController
public class BooksController {
  // One job: Handle HTTP requests/responses
  @GetMapping("/search")
  public ResponseEntity<?> search(
      @RequestParam String query) {
    List<Book> results = bookService.searchBooks(query);
    return ResponseEntity.ok(results);
  }

  // Does NOT contain business logic,
  // does NOT access database directly
}

// 4. AsinGenerator.java - ONLY ASIN generation
public class AsinGenerator {
  // One job: Generate ASINs
  public static String generateFromBook(
      String title, String author) {
    // Hashing logic
  }

  public static boolean isValid(String asin) {
    // Validation logic
  }

  // Does NOT save books, does NOT search,
  // does NOT handle HTTP
}

// 5. DeterministicId.java - ONLY ID generation
public class DeterministicId {
  // One job: Generate deterministic IDs
  public static String forBook(
      String title, String author) {
    // SHA-1 hashing
  }

  // Separate from ASIN generation - different purpose
}
```

What happens when you need to change something:

- **Change ASIN format:** Only edit AsinGenerator
- **Change database:** Only edit BookService
- **Change API format:** Only edit BooksController
- **Add validation:** Create new BookValidator class

Counter-example (what you avoided):

```
// BAD - Everything in one class
public class BooksController {
  @GetMapping("/search")
  public List<Book> search(String query) {
    // HTTP handling (OK here)
    // Business logic (WRONG - should be in service)
    // Database access (WRONG - should be in repository)
    // ASIN generation (WRONG - should be in utility)
    // Email sending (WRONG - should be in email service)
    // ALL IN ONE CLASS - nightmare to maintain!
  }
}
```

Interview Answer Template

Interview Answer:
"Single Responsibility Principle means each class should have only one reason to change. In my bookstore, I separated concerns into different classes: Book class only holds data, BookService only contains business logic, BooksController only handles HTTP requests, and AsinGenerator only generates ASINs. This way, if I need to change the ASIN format, I only touch AsinGenerator - the controller and service stay unchanged. This made my code easier to test and maintain."
Follow-up: "Give a concrete example":
"When I needed to add bulk delete functionality, I only modified BookService to add bulkDeleteBooks() method. The controller just added one new endpoint that calls this method. The Book model didn't change, the ASIN generator didn't change, and other services weren't affected. If everything was in one class, I'd risk breaking search, create, or update operations."

2. OPEN/CLOSED PRINCIPLE (OCP)

Definition

What is OCP?
Software entities should be OPEN for extension but CLOSED for modification. You should be able to add new functionality without changing existing code. Use interfaces, inheritance, and composition.

Why Use It?

Benefits:

- **Safer:** Don't risk breaking working code
- **Flexible:** Easy to add new features
- **Maintainable:** Old code stays stable
- **Testable:** Existing tests still pass

General Example

BAD Example - Modify existing code:

```
public class DiscountCalculator {
  public double calculate(String customerType,
                          double amount) {
    if (customerType.equals("REGULAR")) {
      return amount;
    } else if (customerType.equals("PREMIUM")) {
      return amount * 0.9;  // 10% off
    }
    // Need to ADD GOLD customer? Modify this method!
    else if (customerType.equals("GOLD")) {
      return amount * 0.8;  // 20% off
    }
    // Every new type = modify existing code
    return amount;
  }
}
```

GOOD Example - Extend without modifying:

```
// Interface (open for extension)
interface DiscountStrategy {
  double applyDiscount(double amount);
}

// Implementations (closed for modification)
class RegularDiscount implements DiscountStrategy {
  public double applyDiscount(double amount) {
    return amount;  // No discount
  }
}

class PremiumDiscount implements DiscountStrategy {
  public double applyDiscount(double amount) {
    return amount * 0.9;  // 10% off
  }
}

// Add GOLD without modifying existing classes!
class GoldDiscount implements DiscountStrategy {
  public double applyDiscount(double amount) {
    return amount * 0.8;  // 20% off
  }
}

// Calculator doesn't change when adding new types
class DiscountCalculator {
  public double calculate(DiscountStrategy strategy,
                          double amount) {
    return strategy.applyDiscount(amount);
  }
}
```

In Your Bookstore Project

In Your Bookstore - Repository Pattern:
Interface defines contract (open for extension):

```
// Base interface - never changes
public interface OrderRepository {
  void save(Order order);
  Optional<Order> findById(String orderId);
  List<Order> findByUsername(String username);
}

// Implementation 1: In-Memory (initial)
public class InMemoryOrderRepository
    implements OrderRepository {

  private Map<String, Order> orders = new HashMap<>();

  @Override
  public void save(Order order) {
    orders.put(order.getOrderId(), order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    return Optional.ofNullable(orders.get(orderId));
  }
}

// Implementation 2: DynamoDB (added later)
// EXTENDED without modifying InMemoryOrderRepository!
public class DynamoDbOrderRepository
    implements OrderRepository {

  private DynamoDbTable<Order> orderTable;

  @Override
  public void save(Order order) {
    orderTable.putItem(order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    Key key = Key.builder()
      .partitionValue(orderId).build();
    Order order = orderTable.getItem(key);
    return Optional.ofNullable(order);
  }
}

// Future: Can add MySQLOrderRepository
// WITHOUT changing any existing code!
public class MySQLOrderRepository
    implements OrderRepository {
  // New implementation
}

// CheckoutService NEVER changes!
@Service
public class CheckoutService {
  private final OrderRepository repository;

  public Order checkout(Cart cart, String username) {
    Order order = createOrder(cart, username);
    repository.save(order);  // Works with ALL implementations
    return order;
  }
  // This code is CLOSED for modification
  // but OPEN for extension (new repositories)
}
```

Recommendation System (another example):

```
// Interface for recommendations
public interface RecommendationStrategy {
  List<Book> recommend(User user);
}

// Strategy 1: Genre-based
public class GenreBasedRecommendation
    implements RecommendationStrategy {

  @Override
  public List<Book> recommend(User user) {
    // Recommend books in user's favorite genres
  }
}

// Strategy 2: Collaborative filtering (added later)
public class CollaborativeFiltering
    implements RecommendationStrategy {

  @Override
```

Interview Answer Template

Interview Answer:
"Open/Closed Principle means code should be open for extension but closed for modification. In my bookstore, OrderRepository is an interface with save() and findById() methods. I started with InMemoryOrderRepository for development. When I needed DynamoDB support, I created DynamoDbOrderRepository implementing the same interface. The key is: CheckoutService code never changed - it's closed for modification. But I extended functionality by adding a new implementation - open for extension."
Another example:
"For recommendations, I can add new algorithms like collaborative filtering or content-based filtering by creating new classes implementing RecommendationStrategy interface. The service that uses recommendations doesn't need to change. I'm adding features without modifying existing, tested code."

3. LISKOV SUBSTITUTION PRINCIPLE (LSP)

Definition

What is LSP?
Objects of a subclass should be replaceable with objects of the superclass without breaking the application. Child classes must behave like their parent class. If code works with Parent, it must work with Child.

Why Use It?

Benefits:

- **Reliability:** Substituting implementations doesn't break code
- **Polymorphism:** Can use child anywhere parent is expected
- **Consistency:** Predictable behavior
- **Testability:** Mock objects behave like real ones

General Example

BAD Example - Violates LSP:

```
class Bird {
  public void fly() {
    System.out.println("Flying");
  }
}

class Penguin extends Bird {
  @Override
  public void fly() {
    throw new UnsupportedOperationException(
      "Penguins can't fly!");
  }
  // Violates LSP - Penguin can't do what Bird promises
}

// Code breaks!
void makeBirdFly(Bird bird) {
  bird.fly();  // Works with Bird, breaks with Penguin!
}

Bird myBird = new Penguin();
makeBirdFly(myBird);  // CRASH!
```

GOOD Example - Follows LSP:

```
abstract class Bird {
  abstract void move();
}

class Sparrow extends Bird {
  @Override
  public void move() {
    System.out.println("Flying");
  }
}

class Penguin extends Bird {
  @Override
  public void move() {
    System.out.println("Swimming");
  }
}

// Works with all birds!
void makeBirdMove(Bird bird) {
  bird.move();  // Works with Sparrow AND Penguin
}

Bird bird1 = new Sparrow();
Bird bird2 = new Penguin();
makeBirdMove(bird1);  // "Flying"
makeBirdMove(bird2);  // "Swimming" - both work!
```

In Your Bookstore Project

In Your Bookstore - Repository Substitution:

```
// Parent contract
public interface OrderRepository {
  void save(Order order);
  Optional<Order> findById(String orderId);
}

// Child 1: In-Memory
public class InMemoryOrderRepository
    implements OrderRepository {

  private Map<String, Order> orders = new HashMap<>();

  @Override
  public void save(Order order) {
    if (order == null || order.getOrderId() == null) {
      throw new IllegalArgumentException("Invalid order");
    }
    orders.put(order.getOrderId(), order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    return Optional.ofNullable(orders.get(orderId));
  }
}

// Child 2: DynamoDB
public class DynamoDbOrderRepository
    implements OrderRepository {

  @Override
  public void save(Order order) {
    if (order == null || order.getOrderId() == null) {
      throw new IllegalArgumentException("Invalid order");
    }
    orderTable.putItem(order);
  }

  @Override
  public Optional<Order> findById(String orderId) {
    Key key = Key.builder()
      .partitionValue(orderId).build();
    Order order = orderTable.getItem(key);
    return Optional.ofNullable(order);
  }
}

// Service works with EITHER implementation!
@Service
public class CheckoutService {
  private final OrderRepository repository;

  public Order checkout(Cart cart, String username) {
    Order order = createOrder(cart, username);

    // Works with InMemory OR DynamoDB
    repository.save(order);

    // Works with InMemory OR DynamoDB
    Optional<Order> saved =
      repository.findById(order.getOrderId());

    return saved.orElseThrow();
  }

  // Can substitute ANY OrderRepository implementation
  // without changing this code!
}

// Testing with mock repository (LSP in action)
@Test
public void testCheckout() {
  // Mock repository behaves like real one
  OrderRepository mockRepo = mock(OrderRepository.class);
  when(mockRepo.findById(any()))
    .thenReturn(Optional.of(mockOrder));

  CheckoutService service = new CheckoutService(mockRepo);
  Order result = service.checkout(cart, "user1");

  // Works because mock substitutes real repository
  assertNotNull(result);
}
```

Key Point: Both implementations:

- Accept same inputs (Order objects)
- Return same outputs (Optional<Order>)
- Throw same exceptions (IllegalArgumentException)

Interview Answer Template

Interview Answer:
"Liskov Substitution Principle means you should be able to replace a parent class with any of its child classes without breaking code. In my bookstore, OrderRepository is the parent interface with two children: InMemoryOrderRepository and DynamoDbOrderRepository. The CheckoutService uses OrderRepository type, so I can pass either implementation and it works exactly the same. Both throw the same exceptions for invalid input, both return Optional for null results, and both save orders successfully."
Real example I used:
"During development, I used InMemoryOrderRepository. When writing tests, I used a mock OrderRepository. For production, I'd use DynamoDbOrderRepository. The checkout code never changed - I just substituted different implementations. That's LSP - substituting child classes doesn't break the parent's contract."

4. INTERFACE SEGREGATION PRINCIPLE (ISP)

Definition

What is ISP?
Clients should not be forced to depend on interfaces they don't use. Better to have many small, specific interfaces than one large, general interface. Don't make classes implement methods they don't need.

Why Use It?

Benefits:

- **Flexibility:** Implement only what you need
- **Clarity:** Clear purpose for each interface
- **Maintainability:** Changes to one interface don't affect others
- **No dummy implementations:** Don't implement unused methods

General Example

BAD Example - Fat Interface:

```
// One big interface with everything
interface Worker {
  void work();
  void eat();
  void sleep();
  void getSalary();
  void attendMeeting();
  void submitReport();
}

// Human worker - implements everything
class HumanWorker implements Worker {
  public void work() { /* actual work */ }
  public void eat() { /* eat food */ }
  public void sleep() { /* sleep */ }
  public void getSalary() { /* get paid */ }
  public void attendMeeting() { /* attend */ }
  public void submitReport() { /* submit */ }
}

// Robot worker - forced to implement unused methods!
class RobotWorker implements Worker {
  public void work() { /* actual work */ }

  // Robots don't eat/sleep!
  public void eat() {
    throw new UnsupportedOperationException();
  }
  public void sleep() {
    throw new UnsupportedOperationException();
  }

  public void getSalary() { /* get charged */ }
  public void attendMeeting() { /* attend */ }
  public void submitReport() { /* submit */ }
}
```

GOOD Example - Segregated Interfaces:

```
// Multiple small interfaces
interface Workable {
  void work();
}

interface Payable {
  void getSalary();
}

interface Biological {
  void eat();
  void sleep();
}

interface Reportable {
  void attendMeeting();
  void submitReport();
}

// Human implements what it needs
class HumanWorker implements Workable, Payable,
    Biological, Reportable {
  public void work() { /* work */ }
  public void getSalary() { /* paid */ }
  public void eat() { /* eat */ }
  public void sleep() { /* sleep */ }
  public void attendMeeting() { /* attend */ }
  public void submitReport() { /* submit */ }
}

// Robot implements only what it needs!
class RobotWorker implements Workable, Payable,
    Reportable {
  public void work() { /* work */ }
  public void getSalary() { /* charged */ }
  public void attendMeeting() { /* attend */ }
  public void submitReport() { /* submit */ }
  // No eat/sleep methods - not forced to implement!
}
```

In Your Bookstore Project

In Your Bookstore - Focused Interfaces:
Instead of one huge interface:

```
// BAD - One fat interface
interface BookRepositoryGod {
  // Basic CRUD
  void save(Book book);
  Book findById(String id);
  void delete(String id);

  // Search operations
  List<Book> search(String query);
  List<Book> findByGenre(String genre);
  List<Book> findByAuthor(String author);

  // Analytics (not all repositories need this!)
  int getTotalBooks();
  Map<String, Integer> getBooksByGenre();
  List<Book> getTopSellers();

  // Export (not all repositories need this!)
  String exportToCsv();
  String exportToJson();

  // Email (why is this here?!)
  void emailLowStockAlert();
}

// Implementation forced to implement everything!
class InMemoryBookRepository
    implements BookRepositoryGod {
  // Must implement all 14 methods, even if not needed!
}
```

Your actual implementation (segregated):

```
// Focused interface 1: Basic operations
interface BookRepository {
  void save(Book book);
  Optional<Book> findById(String id);
  void delete(String id);
  List<Book> findAll();
}

// Focused interface 2: Search operations
interface BookSearchRepository {
  List<Book> search(String query);
  List<Book> findByGenre(String genre);
  List<Book> findByAuthor(String author);
}

// Focused interface 3: Bulk operations
interface BulkBookRepository {
  void bulkUpdate(List<BookUpdate> updates);
  void bulkDelete(List<String> ids);
  List<Book> bulkSave(List<Book> books);
}

// Implementations choose what they need
@Service
public class BookService implements
    BookRepository, BookSearchRepository,
    BulkBookRepository {

  // Implements all because it's the main service
  @Override
  public void save(Book book) { ... }

  @Override
  public List<Book> search(String query) { ... }

  @Override
  public void bulkUpdate(List<BookUpdate> updates) { ... }
}

// Different implementation might only need basic ops
public class SimpleBookCache implements BookRepository {
  // Only implements basic CRUD
  @Override
  public void save(Book book) {
    cache.put(book.getId(), book);
  }

  @Override
  public Optional<Book> findById(String id) {
    return Optional.ofNullable(cache.get(id));
  }

  // Doesn't implement search or bulk - not forced to!
}
```

Interview Answer Template

Interview Answer:
"Interface Segregation Principle means having many small, focused interfaces instead of one large interface. In my bookstore, instead of one huge BookRepository interface with 15+ methods, I could create focused interfaces like BookSearchRepository for search operations and BulkBookRepository for bulk operations. This way, a simple caching implementation only needs to implement basic CRUD methods, not search or bulk operations it doesn't use."
Real benefit:
"When I write tests, I can mock just the interface I need. For testing search functionality, I mock BookSearchRepository with 3 methods, not a huge interface with 15 methods. This makes tests simpler and more focused."

5. DEPENDENCY INVERSION PRINCIPLE (DIP)

Definition

What is DIP?
High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces). Abstractions should not depend on details. Details should depend on abstractions. In simple terms: depend on interfaces, not concrete classes.

Why Use It?

Benefits:

- **Loose coupling:** Easy to swap implementations
- **Testability:** Easy to inject mocks
- **Flexibility:** Change dependencies without changing code
- **Maintainability:** Changes are isolated

General Example

BAD Example - Depends on concrete class:

```
// Low-level class
class MySQLDatabase {
  public void save(String data) {
    // MySQL-specific code
  }
}

// High-level class DEPENDS on MySQL directly
class UserService {
  // Tightly coupled to MySQL!
  private MySQLDatabase database = new MySQLDatabase();

  public void saveUser(User user) {
    database.save(user.toString());
  }

  // Can't use PostgreSQL, MongoDB, or test with mock!
}
```

GOOD Example - Depends on abstraction:

```
// Abstraction (interface)
interface Database {
  void save(String data);
}

// Low-level implementations
class MySQLDatabase implements Database {
  public void save(String data) {
    // MySQL code
  }
}

class MongoDatabase implements Database {
  public void save(String data) {
    // MongoDB code
  }
}

// High-level class depends on abstraction
class UserService {
  // Depends on interface, not concrete class!
  private final Database database;

  // Dependency injected through constructor
  public UserService(Database database) {
    this.database = database;
  }

  public void saveUser(User user) {
    database.save(user.toString());
  }

  // Can use MySQL, MongoDB, or mock - flexible!
}

// Usage
Database db = new MySQLDatabase();  // or MongoDatabase
UserService service = new UserService(db);
service.saveUser(user);
```

In Your Bookstore Project

In Your Bookstore - Dependency Injection:
BAD approach (what you avoided):

```java
@RestController
public class BooksController {
  // Creating dependency inside class - BAD!
  private BookService bookService = new BookService();

  @GetMapping
  public List<Book> getAll() {
    return bookService.listAllBooks();
  }

  // Problems:
  // 1. Can't change BookService implementation
  // 2. Can't test with mock service
  // 3. Tightly coupled to concrete class
}
```

GOOD approach (what you did):

```java
// Controller depends on abstraction (interface/service)
@RestController
@RequestMapping("/api/books")
public class BooksController {
  // Depend on interface, not implementation
  private final BookServiceAdapter adapter;

  // Dependency injected by Spring
  @Autowired  // Optional in modern Spring
  public BooksController(BookServiceAdapter adapter) {
    this.adapter = adapter;
  }

  @GetMapping
  public ResponseEntity<List<Book>> getAll() {
    return ResponseEntity.ok(adapter.listAll());
  }

  // Spring automatically injects the right implementation
  // Can test by injecting mock adapter
}

// Service layer also uses DI
@Service
public class BookService {
  // Depends on DynamoDB table abstraction
  private final DynamoDbTable<Book> bookTable;

  public BookService(DynamoDbEnhancedClient enhancedClient) {
    this.bookTable = enhancedClient.table(
      "Books",
      TableSchema.fromBean(Book.class)
    );
  }

  public List<Book> listAllBooks() {
    return bookTable.scan().items().stream()
      .collect(Collectors.toList());
  }
}

// Configuration tells Spring what to inject
@Configuration
public class DynamoDbConfig {
  @Bean
  public DynamoDbEnhancedClient enhancedClient() {
    return DynamoDbEnhancedClient.builder()
      .dynamoDbClient(dynamoDbClient())
      .build();
  }

  @Bean
  public DynamoDbClient dynamoDbClient() {
    return DynamoDbClient.builder()
      .endpointOverride(URI.create("http://localhost:8000"))
      .region(Region.AP_SOUTH_1)
      .build();
  }
}
```

Testing with Dependency Injection:

```java
@Test
public void testGetAllBooks() {
  // Create mock adapter (interface)
  BookServiceAdapter mockAdapter =
    mock(BookServiceAdapter.class);

  // Inject mock into controller
```

Interview Answer Template

Interview Answer:
"Dependency Inversion Principle means depending on abstractions instead of concrete classes. In my bookstore, BooksController doesn't create its own BookService instance. Instead, it depends on BookServiceAdapter interface which is injected through the constructor. Spring framework provides the actual implementation at runtime. This means I can easily inject a mock adapter for testing, or switch to a different implementation without changing controller code."
Concrete example:
"My controller constructor takes BookServiceAdapter as a parameter. During normal operation, Spring injects the real BookService implementation. During testing, I inject a mock that returns fake data. The controller code never changes - it just calls methods on the interface. This is dependency inversion - high-level code (controller) doesn't depend on low-level details (DynamoDB), both depend on abstraction (interface)."

General Example

Simple Example:

```java
// Repository interface
interface UserRepository {
  User findById(String id);
  void save(User user);
  void delete(String id);
  List<User> findAll();
}

// Implementation hides database details
class UserRepositoryImpl implements UserRepository {
  private Connection dbConnection;

  public User findById(String id) {
    // SQL queries hidden here
    ResultSet rs = stmt.executeQuery(
      "SELECT * FROM users WHERE id = '" + id + "'");
    return mapToUser(rs);
  }

  public void save(User user) {
    // SQL hidden here
    stmt.executeUpdate(
      "INSERT INTO users VALUES (...)");
  }
}

// Business logic doesn't see SQL!
class UserService {
  private UserRepository repository;

  public void registerUser(User user) {
    // Business logic only
    if (user.isValid()) {
      repository.save(user);  // No SQL here!
    }
  }
}
```

DESIGN PATTERNS USED

1. REPOSITORY PATTERN

Definition

What is Repository Pattern?
Repository pattern creates an abstraction layer between business logic and data access. It acts like an in-memory collection of objects, hiding database details (SQL, DynamoDB, etc.) from the rest of the application.

Why Use It?

Benefits:

- **Separation:** Business logic doesn't know about database
- **Testability:** Easy to mock data access
- **Flexibility:** Swap databases without changing logic
- **Centralized:** All data access in one place

In Your Bookstore Project

In Your Bookstore:
Architecture:

```
Controller  Service  Repository  Database
(HTTP)      (Logic)  (Data Access) (DynamoDB)
```

Implementation:

```java
// 1. Repository Layer (Data Access)
@Service
public class BookService {
  // This IS your repository
  private final DynamoDbTable<Book> bookTable;

  // Methods act like repository
  public Book getBookById(String id) {
    Key key = Key.builder()
      .partitionValue(id)
      .build();
    return bookTable.getItem(key);
  }

  public void saveBook(Book book) {
    bookTable.putItem(book);
  }

  public List<Book> findAll() {
    return bookTable.scan().items().stream()
      .collect(Collectors.toList());
  }

  // All DynamoDB complexity hidden here!
}

// 2. Service Layer (Business Logic)
@Service
public class CheckoutService {
  private final BookService bookService;
  private final OrderRepository orderRepository;

  public Order checkout(Cart cart, String username) {
    // Business logic
    if (cart.isEmpty()) {
      throw new EmptyCartException();
    }

    // Validate stock (uses repository)
    for (CartItem item : cart.getItems().values()) {
      Book book = bookService.getBookById(
        item.getBookId());
      if (book.getStockQuantity() < item.getQuantity()) {
        throw new InsufficientStockException();
      }
    }

    // Create order
    Order order = new Order(
      UUID.randomUUID().toString(),
      username,
      cart.getItems(),
      cart.calculateTotal()
    );

    // Save via repository
    orderRepository.save(order);

    return order;
  }

  // No DynamoDB code here - all hidden in repositories!
}

// 3. Controller Layer (HTTP Handling)
@RestController
@RequestMapping("/api")
public class CheckoutController {
  private final CheckoutService checkoutService;

  @PostMapping("/cart/checkout")
  public ResponseEntity<?> checkout(
      @RequestBody CheckoutRequest request,
      HttpSession session) {

    // Get cart from session
    Cart cart = (Cart) session.getAttribute("cart");
    String username = (String) session.getAttribute("username");

    // Call service (no database code!)
    Order order = checkoutService.checkout(cart, username);
```

Interview Answer Template

Interview Answer:
"Repository Pattern separates data access from business logic. In my bookstore, BookService acts as a repository - it encapsulates all DynamoDB operations. The CheckoutService doesn't know about DynamoDB at all; it just calls bookService.getBookById() to fetch books. This separation let me test business logic without starting a database, and I could switch from in-memory storage to DynamoDB without changing the checkout code."
Example:
"My controller layer only handles HTTP requests and responses. The service layer contains business rules like stock validation and order creation. The repository layer (BookService) handles all database operations. Each layer has one job, and layers only talk to their neighbors. This is the Repository Pattern in action."

2. FACTORY PATTERN

Definition

What is Factory Pattern?
Factory Pattern provides a way to create objects without specifying the exact class. A factory method or class handles object creation logic, hiding complexity from the caller. Instead of `new ClassName()`, you call `Factory.create()`.

Why Use It?

Benefits:

- **Centralized creation:** One place to manage object creation
- **Encapsulation:** Hide complex initialization
- **Flexibility:** Easy to change creation logic
- **Consistency:** Ensure objects created correctly

General Example

Simple Example - Shape Factory:

```java
// Without factory - client creates directly
Circle c = new Circle(5);
Rectangle r = new Rectangle(10, 20);
// Client knows about concrete classes

// With factory
class ShapeFactory {
  public static Shape createShape(String type) {
    switch (type) {
      case "CIRCLE":
        return new Circle(5);  // Default size
      case "RECTANGLE":
        return new Rectangle(10, 20);
      case "TRIANGLE":
        return new Triangle(3, 4, 5);
      default:
        throw new IllegalArgumentException(
          "Unknown shape: " + type);
    }
  }
}

// Client code is simple
Shape s1 = ShapeFactory.createShape("CIRCLE");
Shape s2 = ShapeFactory.createShape("RECTANGLE");
// Client doesn't know about constructors!
```

In Your Bookstore Project

In Your Bookstore - ASIN Generator:

```java
// Factory for generating ASINs
public class AsinGenerator {
  private static final String CHARS =
    "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  private static final SecureRandom random =
    new SecureRandom();

  // Factory method 1: Random ASIN
  public static String generateRandom() {
    StringBuilder sb = new StringBuilder("B0");
    for (int i = 0; i < 8; i++) {
      int index = random.nextInt(CHARS.length());
      sb.append(CHARS.charAt(index));
    }
    return sb.toString();
  }

  // Factory method 2: Deterministic ASIN
  public static String generateFromBook(
      String title, String author) {

    // Complex logic hidden from caller
    String input = (title + "|" + author)
      .toLowerCase()
      .trim();

    byte[] hash = sha256(input);

    StringBuilder sb = new StringBuilder("B0");
    for (int i = 0; i < 8; i++) {
      int index = Math.abs(hash[i]) % CHARS.length();
      sb.append(CHARS.charAt(index));
    }

    return sb.toString();
  }

  // Factory method 3: Validate ASIN
  public static boolean isValid(String asin) {
    return asin != null
      && asin.length() == 10
      && asin.startsWith("B0")
      && asin.matches("[A-Z0-9]{10}");
  }

  private static byte[] sha256(String input) {
    // SHA-256 hashing logic
  }
}

// Usage in Book class
public class Book {
  private String asin;

  public void ensureAsin() {
    if (asin == null || asin.isBlank()) {
      // Factory creates ASIN
      this.asin = AsinGenerator.generateFromBook(
        this.title,
        this.author
      );
    }
  }
}

// Usage in BookService
public Book saveBook(Book book) {
  // Factory ensures ASIN exists
  book.ensureAsin();

  // Validate using factory
  if (!AsinGenerator.isValid(book.getAsin())) {
    throw new InvalidAsinException();
  }

  bookTable.putItem(book);
  return book;
}
```

Another Factory - DeterministicId:

```java
// Factory for generating deterministic IDs
public class DeterministicId {

  public static String forBook(
      String title, String author) {

    // Complex logic encapsulated
    String input = (title + "|" + author)
      .toLowerCase()
```

Interview Answer Template

Interview Answer:

"Factory Pattern centralizes object creation logic. In my bookstore, I created AsinGenerator as a factory class with static methods. Instead of each part of the code creating ASINs differently, they all call AsinGenerator.generateFromBook(title, author). This factory handles the complex SHA-256 hashing and formatting, returning a clean ASIN like 'B0A1B2C3D4'. If I need to change the ASIN format or algorithm, I only modify the factory - all callers automatically get the new format."

Another example:

"I also used factory pattern for deterministic IDs with DeterministicId.forBook(). This factory ensures the same book always gets the same ID, preventing duplicates. The complex SHA-1 hashing and hex conversion are hidden inside the factory. Callers just get a clean ID back."

3. SINGLETON PATTERN

Definition

What is Singleton Pattern?

Singleton ensures a class has only ONE instance throughout the application, and provides a global point of access to it. Used for shared resources like database connections, configuration, caches.

Why Use It?

Benefits:

- **Resource efficiency:** One instance, not thousands
- **Shared state:** All parts of app use same instance
- **Global access:** Easy to access from anywhere
- **Controlled initialization:** Lazy or eager loading

General Example

Manual Singleton Implementation:

```java
public class DatabaseConnection {
  // Single instance (static)
  private static DatabaseConnection instance;

  // Private constructor - can't create from outside!
  private DatabaseConnection() {
    // Initialize database connection
  }

  // Public method to get the single instance
  public static DatabaseConnection getInstance() {
    if (instance == null) {
              // Create instance only once
      instance = new DatabaseConnection();
    }
    return instance;
  }

  public void query(String sql) {
    // Use the connection
  }
}

// Usage - always returns same instance
DatabaseConnection db1 = DatabaseConnection.getInstance();
DatabaseConnection db2 = DatabaseConnection.getInstance();
// db1 == db2 (same object!)
```

Thread-Safe Singleton:

```java
public class Configuration {
  // Eager initialization (created at class loading)
  private static final Configuration INSTANCE =
    new Configuration();

  private String apiKey;
  private String dbUrl;

  private Configuration() {
    // Load configuration
    this.apiKey = loadApiKey();
    this.dbUrl = loadDbUrl();
  }

  public static Configuration getInstance() {
    return INSTANCE;  // Always returns same instance
  }

  public String getApiKey() { return apiKey; }
}
```

In Your Bookstore Project

In Your Bookstore - Spring Singletons:
You didn't write Singleton manually - Spring did it!

```
// Spring creates ONE instance of each @Service
@Service
public class BookService {
  // Spring ensures only one instance exists
  // All controllers get the SAME BookService
}

// Spring creates ONE instance of each @RestController
@RestController
public class BooksController {
  // Only one controller instance handles all requests
  private final BookService bookService;

  public BooksController(BookService bookService) {
    // Spring injects the singleton BookService
    this.bookService = bookService;
  }
}

// Configuration beans are also singletons
@Configuration
public class DynamoDbConfig {

  @Bean  // Spring creates ONE instance
  public DynamoDbClient dynamoDbClient() {
    return DynamoDbClient.builder()
      .endpointOverride(
        URI.create("http://localhost:8000"))
      .region(Region.AP_SOUTH_1)
      .build();
    // This expensive object created only once!
  }

  @Bean  // Spring creates ONE instance
  public DynamoDbEnhancedClient enhancedClient(
      DynamoDbClient client) {
    return DynamoDbEnhancedClient.builder()
      .dynamoDbClient(client)
      .build();
  }
}

// All services share the SAME DynamoDB client
@Service
public class BookService {
  private final DynamoDbEnhancedClient client;
  // Injected singleton
}

@Service
public class UserService {
  private final DynamoDbEnhancedClient client;
  // Same singleton as BookService!
}
```

How Spring Singleton works:

```
// When Spring starts:
// 1. Creates ONE BookService instance
BookService bookService = new BookService();

// 2. Creates ONE BooksController instance
BooksController controller1 =
  new BooksController(bookService);

// 3. Creates ONE AdminController instance
AdminController controller2 =
  new AdminController(bookService);

// controller1 and controller2 share SAME bookService!

// When request comes:
// Request 1  controller1.getAll()
//              uses shared bookService
// Request 2  controller1.search()
//              uses SAME bookService
// Request 3  controller2.createBook()
//              uses SAME bookService
```

Why Singleton matters in your project:

1. **DynamoDB Client:** Expensive to create, reused for all requests
2. **Services:** Shared state (in-memory books), one instance
3. **Controllers:** Handle multiple requests, one instance per

Interview Answer Template

Interview Answer:
"Singleton Pattern ensures only one instance of a class exists. In my bookstore, Spring framework automatically makes all @Service and @RestController beans singletons. For example, there's only one BookService instance shared by all controllers and all user requests. This is efficient because creating DynamoDB clients and service objects is expensive - doing it once and reusing is much better than creating new instances for every request."
Example:
"My DynamoDB client is a singleton bean created in configuration. This client manages connection pooling and is expensive to initialize. By making it a singleton, Spring creates it once at startup, and all services (BookService, UserService, OrderService) share the same client. This saves memory and improves performance."
When NOT to use Singleton:
"Shopping carts are NOT singletons - each user needs their own cart. I store carts in HttpSession, so User1 and User2 have separate Cart instances. If Cart was a singleton, all users would share the same cart, which would be a bug!"

General Example

Simple Example - Media Player:

```
// Old interface
interface MediaPlayer {
  void play(String filename);
}

// New advanced interface (incompatible)
interface AdvancedMediaPlayer {
  void playVlc(String filename);
  void playMp4(String filename);
}

// Adapter bridges the gap
class MediaAdapter implements MediaPlayer {
  private AdvancedMediaPlayer advancedPlayer;

  public MediaAdapter(String audioType) {
    if (audioType.equals("vlc")) {
      advancedPlayer = new VlcPlayer();
    } else if (audioType.equals("mp4")) {
      advancedPlayer = new Mp4Player();
    }
  }

  @Override
  public void play(String filename) {
    // Adapts simple play() to advanced interface
    if (filename.endsWith(".vlc")) {
      advancedPlayer.playVlc(filename);
    } else if (filename.endsWith(".mp4")) {
      advancedPlayer.playMp4(filename);
    }
  }
}

// Client uses simple interface
MediaPlayer player = new MediaAdapter("mp4");
player.play("video.mp4");  // Works!
```

4. ADAPTER PATTERN

Definition

What is Adapter Pattern?
Adapter Pattern allows incompatible interfaces to work together. It acts as a bridge between two incompatible interfaces. Think of it like a power adapter that lets you plug a US device into a European outlet.

Why Use It?

Benefits:

- **Compatibility:** Connect incompatible systems
- **Reusability:** Use existing code without modification
- **Separation:** Keep different systems decoupled
- **Flexibility:** Easy to swap implementations

In Your Bookstore Project

In Your Bookstore - BookServiceAdapter:
The Problem:

```
// BookService returns complex DynamoDB types
@Service
public class BookService {
  public Book getBookById(String id) {
    // Returns Book or null
    return bookTable.getItem(key);
  }

  public List<Book> searchBooks(String query) {
    // Returns List (could be empty)
    return filteredBooks;
  }
}

// Spring controllers expect Optional, ResponseEntity
@RestController
public class BooksController {
  @GetMapping("/{id}")
  public ResponseEntity<?> getBook(@PathVariable String id) {
    Book book = bookService.getBookById(id);

    // Need to handle null manually
    if (book == null) {
      return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(book);

    // Repetitive null checks everywhere!
  }
}
```

The Solution - Adapter:

```
// Adapter wraps BookService
@Component
public class BookServiceAdapter {
  private final BookService bookService;

  public BookServiceAdapter(BookService bookService) {
    this.bookService = bookService;
  }

  // Adapts null  Optional
  public Optional<Book> findById(String id) {
    Book book = bookService.getBookById(id);
    return Optional.ofNullable(book);
  }

  // Adapts complex query  simple list
  public List<Book> search(String query) {
    return bookService.searchBooks(query);
  }

  // Adapts save operation
  public Book save(Book book) {
    return bookService.saveBook(book);
  }

  // Adapts list all operation
  public List<Book> listAll() {
    return bookService.listAllBooks();
  }

  // Adapts ASIN lookup
  public Optional<Book> findByAsin(String asin) {
    Book book = bookService.getBookByAsin(asin);
    return Optional.ofNullable(book);
  }
}

// Controller now uses adapter (cleaner!)
@RestController
public class BooksController {
  private final BookServiceAdapter adapter;

  public BooksController(BookServiceAdapter adapter) {
    this.adapter = adapter;
  }

  @GetMapping("/{id}")
  public ResponseEntity<?> getBook(@PathVariable String id) {
    // Adapter returns Optional - cleaner!
    return adapter.findById(id)
      .map(ResponseEntity::ok)
      .orElse(ResponseEntity.notFound().build());

    // One line instead of if-else!
  }
```

Interview Answer Template

Interview Answer:
"Adapter Pattern bridges incompatible interfaces. In my bookstore, BookService returns Book objects that might be null, but Spring controllers work better with Optional types. I created BookServiceAdapter that wraps the service and converts null returns to Optional. This made controller code much cleaner - instead of if-else null checks, I use Optional.map() and orElse() methods. The adapter acts as a bridge between the service layer and controller layer."
Concrete benefit:
"Before the adapter, every controller method had 3-4 lines of null checking. After adding the adapter, controller methods became one-liners using Optional methods. The adapter also makes it easy to add features like caching or logging without modifying the service or controllers."

5. STRATEGY PATTERN

Definition

What is Strategy Pattern?
Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. The algorithm can vary independently from clients that use it. Allows selecting algorithm at runtime.

Why Use It?

Benefits:

- **Flexibility:** Switch algorithms at runtime
- **Maintainability:** Each algorithm in separate class
- **Testability:** Test each strategy independently
- **No conditionals:** Avoid large if-else or switch statements

General Example

Simple Example - Sorting Strategies:

```
// Strategy interface
interface SortStrategy {
  void sort(int[] array);
}

// Strategy 1: Bubble Sort
class BubbleSortStrategy implements SortStrategy {
  public void sort(int[] array) {
    // Bubble sort implementation
    for (int i = 0; i < array.length; i++) {
      for (int j = 0; j < array.length - 1; j++) {
        if (array[j] > array[j + 1]) {
          // Swap
        }
      }
    }
  }
}

// Strategy 2: Quick Sort
class QuickSortStrategy implements SortStrategy {
  public void sort(int[] array) {
    // Quick sort implementation
    quickSort(array, 0, array.length - 1);
  }
}

// Context uses strategy
class Sorter {
  private SortStrategy strategy;

  public void setStrategy(SortStrategy strategy) {
    this.strategy = strategy;
  }

  public void sort(int[] array) {
    strategy.sort(array);  // Delegates to strategy
  }
}

// Usage - switch strategies
Sorter sorter = new Sorter();

sorter.setStrategy(new BubbleSortStrategy());
sorter.sort(smallArray);  // Uses bubble sort

sorter.setStrategy(new QuickSortStrategy());
sorter.sort(largeArray);  // Uses quick sort
```

In Your Bookstore Project

In Your Bookstore - Sorting Strategies: Implementation:

```
// In BooksController - implicit strategy pattern
@GetMapping
public ResponseEntity<List<Book>> getAllBooks(
    @RequestParam(defaultValue = "title") String sortBy,
    @RequestParam(defaultValue = "asc") String order) {

  List<Book> books = adapter.listAll();

  // Strategy selected based on parameter
  Comparator<Book> comparator;

  switch (sortBy.toLowerCase()) {
    case "price":
      comparator = Comparator.comparing(Book::getPrice);
      break;
    case "author":
      comparator = Comparator.comparing(Book::getAuthor);
      break;
    case "stock":
      comparator = Comparator.comparing(Book::getStockQuantity);
      break;
    case "title":
    default:
      comparator = Comparator.comparing(Book::getTitle);
      break;
  }

  // Apply sort direction
  if ("desc".equalsIgnoreCase(order)) {
    comparator = comparator.reversed();
  }

  // Execute strategy
  books.sort(comparator);

  return ResponseEntity.ok(books);
}
```

Better approach (explicit strategies):

```
// Strategy interface
interface BookSortStrategy {
  Comparator<Book> getComparator();
}

// Strategy 1: Sort by price
class PriceSortStrategy implements BookSortStrategy {
  private final boolean ascending;

  public PriceSortStrategy(boolean ascending) {
    this.ascending = ascending;
  }

  @Override
  public Comparator<Book> getComparator() {
    Comparator<Book> comp =
      Comparator.comparing(Book::getPrice);
    return ascending ? comp : comp.reversed();
  }
}

// Strategy 2: Sort by title
class TitleSortStrategy implements BookSortStrategy {
  private final boolean ascending;

  public TitleSortStrategy(boolean ascending) {
    this.ascending = ascending;
  }

  @Override
  public Comparator<Book> getComparator() {
    Comparator<Book> comp =
      Comparator.comparing(Book::getTitle);
    return ascending ? comp : comp.reversed();
  }
}

// Strategy 3: Sort by author
class AuthorSortStrategy implements BookSortStrategy {
  @Override
  public Comparator<Book> getComparator() {
    return Comparator.comparing(Book::getAuthor);
  }
}

// Factory to create strategies
class BookSortStrategyFactory {
  public static BookSortStrategy getStrategy(
```

Interview Answer Template

Interview Answer:
"Strategy Pattern lets you choose algorithms at runtime. In my bookstore, I used it for sorting books. Users can sort by price, title, author, or stock, and choose ascending or descending order. Instead of one giant if-else block, I use Java's Comparator interface - each sort criterion is a different strategy. The controller selects the appropriate Comparator based on query parameters and applies it to the book list."
Example:
"When a user requests /api/books?sortBy=price&order=desc, the controller creates a price comparator with reversed order. For /api/books?sortBy=title&order=asc, it creates a title comparator with normal order. The sorting logic is the same (books.sort()), but the strategy (comparator) changes. This is Strategy Pattern - the algorithm (comparator) is interchangeable."

6. BUILDER PATTERN

Definition

What is Builder Pattern?
Builder Pattern constructs complex objects step by step. It separates object construction from representation, allowing the same construction process to create different representations. Useful for objects with many optional parameters.

Why Use It?

Benefits:

- **Readable:** Fluent API with method chaining
- **Flexible:** Set only needed parameters
- **Immutable:** Build once, no setters needed
- **Validation:** Validate in build() method

General Example

Simple Example - Without Builder (messy):

```java
// Constructor with many parameters
public class User {
  private String username;
  private String email;
  private String firstName;
  private String lastName;
  private int age;
  private String phone;

  // Too many parameters - hard to remember order!
  public User(String username, String email,
              String firstName, String lastName,
              int age, String phone) {
    this.username = username;
    this.email = email;
    // ...
  }
}

// Usage - confusing!
User user = new User(
  "john",
  "john@example.com",
  "John",
  "Doe",
  25,
  "123-456-7890"
);
// What does each parameter mean?
```

With Builder (clean):

```java
public class User {
  private final String username;
  private final String email;
  private final String firstName;
  private final String lastName;
  private final int age;
  private final String phone;

  // Private constructor
  private User(Builder builder) {
    this.username = builder.username;
    this.email = builder.email;
    this.firstName = builder.firstName;
    this.lastName = builder.lastName;
    this.age = builder.age;
    this.phone = builder.phone;
  }

  // Builder class
  public static class Builder {
    private String username;
    private String email;
    private String firstName;
    private String lastName;
    private int age;
    private String phone;

    public Builder username(String username) {
      this.username = username;
      return this;  // Return this for chaining
    }

    public Builder email(String email) {
      this.email = email;
      return this;
    }

    public Builder firstName(String firstName) {
      this.firstName = firstName;
      return this;
    }

    public Builder age(int age) {
      this.age = age;
      return this;
    }

    // Optional parameters
    public Builder phone(String phone) {
      this.phone = phone;
      return this;
    }

    public User build() {
      // Validation
      if (username == null || email == null) {
        throw new IllegalStateException(
          "Username and email required");
```

In Your Bookstore Project

In Your Bookstore - DynamoDB SDK Builders: You used builders from AWS SDK:

```java
// Building DynamoDB Key
Key key = Key.builder()
  .partitionValue(bookId)
  .build();

// Building Query Request
QueryEnhancedRequest request =
  QueryEnhancedRequest.builder()
    .queryConditional(
      QueryConditional.keyEqualTo(key))
    .limit(10)
    .scanIndexForward(true)
    .build();

// Building DynamoDB Client
DynamoDbClient client = DynamoDbClient.builder()
  .endpointOverride(
    URI.create("http://localhost:8000"))
  .region(Region.AP_SOUTH_1)
  .credentialsProvider(
    StaticCredentialsProvider.create(
      AwsBasicCredentials.create("dummy", "dummy")))
  .build();

// Building Enhanced Client
DynamoDbEnhancedClient enhancedClient =
  DynamoDbEnhancedClient.builder()
    .dynamoDbClient(client)
    .extensions(Collections.singletonList(
      AttributeConverterProvider.defaultProvider()))
    .build();
```

Why builders are better:

```java
// Without builder (ugly constructor)
DynamoDbClient client = new DynamoDbClient(
  URI.create("http://localhost:8000"),
  Region.AP_SOUTH_1,
  StaticCredentialsProvider.create(
    AwsBasicCredentials.create("dummy", "dummy")),
  null,  // Optional parameter 1
  null,  // Optional parameter 2
  null,  // Optional parameter 3
  true,  // What is this?
  false, // What is this?
  30000  // Timeout? Port? Who knows!
);

// With builder (clear and readable)
DynamoDbClient client = DynamoDbClient.builder()
  .endpointOverride(
    URI.create("http://localhost:8000"))
  .region(Region.AP_SOUTH_1)
  .credentialsProvider(
    StaticCredentialsProvider.create(
      AwsBasicCredentials.create("dummy", "dummy")))
  .build();
// Each parameter is labeled - much clearer!
```

Custom Builder Example (Order):

```java
// Order could use builder pattern
public class Order {
  private final String orderId;
  private final String username;
  private final Map<String, CartItem> items;
  private final BigDecimal total;
  private final Instant createdAt;
  private final String status;
  private final String shippingAddress;

  private Order(Builder builder) {
    this.orderId = builder.orderId;
    this.username = builder.username;
    this.items = builder.items;
    this.total = builder.total;
    this.createdAt = builder.createdAt;
    this.status = builder.status;
    this.shippingAddress = builder.shippingAddress;
  }

  public static class Builder {
    private String orderId;
    private String username;
    private Map<String, CartItem> items;
    private BigDecimal total;
```

Interview Answer Template

Interview Answer:

"Builder Pattern creates complex objects step-by-step with a fluent API. In my bookstore, I extensively used builders from the AWS DynamoDB SDK. For example, to create a DynamoDB client, I use DynamoDbClient.builder() and chain methods like .endpointOverride(), .region(), and .credentialsProvider() before calling .build(). This is much more readable than a constructor with 10 parameters where you don't know what each value means."

Example:

"When querying DynamoDB, I build a Key object with Key.builder().partitionValue(id).build(). Then I build a QueryEnhancedRequest with .queryConditional(), .limit(), and other optional parameters. The builder pattern makes it clear what each parameter does, allows me to skip optional parameters, and validates everything in the build() method. It's like constructing an object piece by piece instead of all at once."

DATA STRUCTURES IN DEPTH

1. HashMap (Shopping Cart)

Definition

What is HashMap?

HashMap is a data structure that stores key-value pairs. It uses hashing to compute an index where values are stored, providing O(1) average-time complexity for insert, delete, and lookup operations.

Why Use It?

When to use HashMap:

- Need fast lookup by key (O(1))
- Keys are unique
- Order doesn't matter (or use LinkedHashMap for order)
- Frequent additions/removals

General Example

How HashMap works internally:

```
// HashMap uses array of buckets
// Hash function converts key to index
int index = hash(key) % array.length;

// Example
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 25);  // hash("Alice")  index 3
ages.put("Bob", 30);    // hash("Bob")  index 7
ages.put("Charlie", 35); // hash("Charlie")  index 3

// Collision at index 3 - uses linked list
// Bucket 3: [Alice:25]  [Charlie:35]

// Lookup
int age = ages.get("Alice");  // hash("Alice")  3
                              // Check bucket 3
                              // Find "Alice"  return 25
                              // O(1) average time!
```

Time Complexity:

- **Insert:** O(1) average, O(n) worst case
- **Get:** O(1) average, O(n) worst case
- **Delete:** O(1) average, O(n) worst case
- **Contains:** O(1) average

In Your Bookstore Project

In Your Bookstore - Cart Implementation:

```
public class Cart {
  // LinkedHashMap preserves insertion order
  // Key = bookId, Value = CartItem
  private final Map<String, CartItem> items =
    new LinkedHashMap<>();

  // Add book - O(1)
  public synchronized void addBook(
      Book book, int quantity) {

    String key = book.getId();  // Key is book ID

    // Check if book already in cart - O(1)
    if (items.containsKey(key)) {
      // Update existing item
      CartItem existing = items.get(key);  // O(1)
      existing.incrementQuantity(quantity);
    } else {
      // Add new item - O(1)
      CartItem newItem = new CartItem(
        book.getId(),
        book.getTitle(),
        book.getAuthor(),
        book.getIsbn(),
        book.getPrice(),
        quantity
      );
      items.put(key, newItem);  // O(1)
    }
  }

  // Remove book - O(1)
  public synchronized void removeBook(String bookId) {
    items.remove(bookId);  // O(1)
  }

  // Update quantity - O(1)
  public synchronized void updateQuantity(
      String bookId, int quantity) {

    if (items.containsKey(bookId)) {  // O(1)
      CartItem item = items.get(bookId);  // O(1)
      item.setQuantity(quantity);
    }
  }

  // Calculate total - O(n) where n = items
  public synchronized BigDecimal calculateTotal() {
    return items.values().stream()
      .map(CartItem::totalPrice)
      .reduce(BigDecimal.ZERO, BigDecimal::add);
  }

  // Get all items - O(1) to return map
  public Map<String, CartItem> getItems() {
    return Collections.unmodifiableMap(items);
  }

  // Check if empty - O(1)
  public boolean isEmpty() {
    return items.isEmpty();
  }

  // Get size - O(1)
  public int size() {
    return items.size();
  }

  // Clear cart - O(n)
  public synchronized void clear() {
    items.clear();
  }
}
```

Why HashMap for cart:

1. **Fast lookup:** Check if book in cart - O(1)
2. **Unique books:** Each book appears once (key = book ID)
3. **Easy update:** Modify quantity without searching
4. **Efficient removal:** Remove by book ID - O(1)

HashMap vs ArrayList for cart:

```
// With ArrayList (BAD for cart)
List<CartItem> items = new ArrayList<>();

// Add - need to search first - O(n)
```

Interview Answer Template

Interview Answer:
"I used HashMap for the shopping cart because it provides O(1) lookup, insert, and delete operations. The key is the book ID, and the value is the CartItem with quantity and price. When a user adds a book to the cart, I can instantly check if it's already there with containsKey() - O(1) operation. If it exists, I update the quantity; if not, I add a new entry. This is much faster than using an ArrayList where I'd have to loop through all items to check if a book exists - that would be O(n)."
Follow-up: "Why LinkedHashMap instead of HashMap?":
"I used LinkedHashMap which extends HashMap but also maintains insertion order. This means when users view their cart, books appear in the order they were added, which feels more natural. Regular HashMap doesn't guarantee any order. The performance is still O(1) for all operations, but LinkedHashMap uses slightly more memory to maintain the linked list of entries."

2. LinkedList (Browsing History)

Definition

What is LinkedList?
LinkedList is a data structure where each element (node) contains data and a reference to the next (and optionally previous) node. It provides O(1) insertion/deletion at ends but O(n) random access.

Why Use It?

When to use LinkedList:
- Frequent insertions/deletions at beginning or end
- Don't need random access by index
- Implementing queues or stacks
- Need to maintain insertion order

General Example

How LinkedList works:

```
// LinkedList structure
class Node {
  String data;
  Node next;
  Node prev;  // Doubly linked
}

// Example
LinkedList<String> list = new LinkedList<>();

// Add to front - O(1)
list.addFirst("A");
// [A]

list.addFirst("B");
// [B]  [A]

// Add to end - O(1)
list.addLast("C");
// [B]  [A]  [C]

// Remove from front - O(1)
list.removeFirst();  // Removes "B"
// [A]  [C]

// Remove from end - O(1)
list.removeLast();  // Removes "C"
// [A]

// Get by index - O(n) - need to traverse
String item = list.get(5);  // Slow!
// Start at head, follow 5 next pointers

// Remove by value - O(n) - need to search
list.remove("A");  // Need to find "A" first
```

Time Complexity:
- **addFirst/addLast:** O(1)
- **removeFirst/removeLast:** O(1)
- **get(index):** O(n)
- **remove(value):** O(n)
- **contains(value):** O(n)

In Your Bookstore Project

In Your Bookstore - Browsing History:

```
@Service
public class BrowsingHistoryService {
  // Maximum history size
  private static final int MAX_CAPACITY = 10;

  // Store user histories
  private final Map<String, LinkedList<String>> histories
    = new HashMap<>();

  // Add book to history
  public void addBookToHistory(
      String username, String bookId) {

    // Get user's history (or create new)
    LinkedList<String> history =
      histories.computeIfAbsent(
        username,
        k -> new LinkedList<>());

    // Remove if already exists (avoid duplicates)
    // O(n) but n  10, so effectively O(1)
    history.remove(bookId);

    // Add to front (most recent)
    // O(1) operation
    history.addFirst(bookId);

    // Maintain capacity limit
    // O(1) operation
    while (history.size() > MAX_CAPACITY) {
      history.removeLast();  // Remove oldest
    }

    // Save to DynamoDB (simplified)
    saveHistoryToDynamoDB(username, history);
  }

  // Get user's history
  public List<String> getHistory(String username) {
    LinkedList<String> history =
      histories.getOrDefault(
        username,
        new LinkedList<>());

    // Return copy to prevent modification
    return new ArrayList<>(history);
  }

  // Get recently viewed books with details
  public List<Book> getRecentBooks(
      String username, BookService bookService) {

    List<String> bookIds = getHistory(username);

    // Fetch book details
    return bookIds.stream()
      .map(bookService::getBookById)
      .filter(Objects::nonNull)
      .collect(Collectors.toList());
  }
}
```

Why LinkedList for history:
1. **FIFO with capacity:** Most recent at front, oldest removed from end
2. **Efficient adds:** addFirst() is O(1)
3. **Efficient removes:** removeLast() is O(1)
4. **Order matters:** Show books in chronological order
5. **Small size:** Max 10 items, so O(n) operations are fast

LinkedList vs ArrayList for history:

```
// With ArrayList (less efficient)
ArrayList<String> history = new ArrayList<>();

// Add to front - O(n) - shifts all elements!
history.add(0, bookId);  // Expensive!

// Remove from end - O(1)
if (history.size() > MAX_CAPACITY) {
  history.remove(history.size() - 1);
}

// With LinkedList (efficient)
LinkedList<String> history = new LinkedList<>();
```

Interview Answer Template

Interview Answer:
"I used LinkedList for browsing history because it excels at adding elements to the front and removing from the end - both O(1) operations. When a user views a book, I add its ID to the front of their history with addFirst(). If the history exceeds 10 books, I remove the oldest one with removeLast(). This maintains a FIFO (First In, First Out) structure with a capacity limit. LinkedList is perfect for this because these operations are constant time, unlike ArrayList where adding to the front would require shifting all elements - O(n) operation."
Follow-up: "Why not ArrayList?":
"ArrayList would be inefficient here. Adding to index 0 with add(0, item) shifts all existing elements right - that's O(n). With LinkedList, addFirst() just updates pointers - O(1). Since browsing history has frequent adds to the front and removes from the end, LinkedList is the optimal choice. The only downside is LinkedList uses more memory per element (stores prev/next pointers), but with max 10 items, this is negligible."

3. ArrayList (Book Lists)

Definition

What is ArrayList?
ArrayList is a resizable array implementation. It provides fast random access by index (O(1)) but slower insertions/deletions in the middle (O(n)). Backed by an array that grows dynamically.

Why Use It?

When to use ArrayList:

- Need fast random access by index
- Frequent reads, infrequent writes
- Iterate through all elements
- Don't need frequent insertions/deletions in middle

General Example

How ArrayList works:

```
// ArrayList backed by array
class ArrayList<E> {
  private Object[] elements;
  private int size;

  // Get by index - O(1)
  public E get(int index) {
    return (E) elements[index];  // Direct array access
  }

  // Add to end - O(1) amortized
  public boolean add(E element) {
    if (size == elements.length) {
      resize();  // Occasional O(n)
    }
    elements[size++] = element;
    return true;
  }

  // Add at index - O(n) - need to shift
  public void add(int index, E element) {
    // Shift elements right
    for (int i = size; i > index; i--) {
      elements[i] = elements[i - 1];
    }
    elements[index] = element;
    size++;
  }

  // Remove - O(n) - need to shift
  public E remove(int index) {
    E removed = (E) elements[index];
    // Shift elements left
    for (int i = index; i < size - 1; i++) {
      elements[i] = elements[i + 1];
    }
    size--;
    return removed;
  }
}
```

Time Complexity:

- **get(index):** O(1)
- **add(element):** O(1) amortized
- **add(index, element):** O(n)
- **remove(index):** O(n)
- **contains(value):** O(n)

In Your Bookstore Project

In Your Bookstore - Search Results Lists:

```
// BookService - search returns ArrayList
public List<Book> searchBooks(String query) {
  // Scan all books
  PageIterable<Book> pages = bookTable.scan();
  List<Book> allBooks = pages.items().stream()
    .collect(Collectors.toList());  // ArrayList

  String lowerQuery = query.toLowerCase();

  // Filter books
  return allBooks.stream()
    .filter(b ->
      b.getTitle().toLowerCase().contains(lowerQuery) ||
      b.getAuthor().toLowerCase().contains(lowerQuery))
    .collect(Collectors.toList());  // Returns ArrayList
}

// Controller - pagination with ArrayList
@GetMapping
public ResponseEntity<Map<String, Object>> getAllBooks(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "12") int size) {

  List<Book> allBooks = adapter.listAll();  // ArrayList

  // Calculate pagination - uses ArrayList.size() - O(1)
  int totalItems = allBooks.size();
  int totalPages = (int) Math.ceil(
    (double) totalItems / size);

  // Get sublist - uses ArrayList.subList() - O(1)
  int start = page * size;
  int end = Math.min(start + size, totalItems);
  List<Book> pageBooks = allBooks.subList(start, end);

  return ResponseEntity.ok(Map.of(
    "books", pageBooks,
    "currentPage", page,
    "totalPages", totalPages,
    "totalItems", totalItems
  ));
}

// Sorting with ArrayList
@GetMapping
public List<Book> getSortedBooks(
    @RequestParam String sortBy) {

  List<Book> books = adapter.listAll();  // ArrayList

  // Sort in-place - O(n log n)
  switch (sortBy) {
    case "price":
      books.sort(Comparator.comparing(Book::getPrice));
      break;
    case "title":
      books.sort(Comparator.comparing(Book::getTitle));
      break;
  }

  return books;
}

// Filtering with ArrayList
public List<Book> filterByGenre(String genre) {
  List<Book> allBooks = bookService.listAllBooks();

  // Filter - creates new ArrayList
  return allBooks.stream()
    .filter(b -> genre.equals(b.getGenre()))
    .collect(Collectors.toList());
}
```

Why ArrayList for book lists:

1. **Random access:** Pagination needs get(index) - O(1)
2. **Iteration:** Frequently loop through all books
3. **Sorting:** sort() method works well with ArrayList
4. **Size operations:** size() is O(1)
5. **Sublist:** subList() for pagination is O(1)

ArrayList vs LinkedList for book lists:

```
// Pagination with ArrayList (GOOD)
List<Book> books = new ArrayList<>(allBooks);
List<Book> page = books.subList(start, end);  // O(1)
```

4. HashSet (Unique Collections)

Definition

What is HashSet?
HashSet is a collection that stores unique elements with no duplicates. It uses HashMap internally where elements are keys and dummy values. Provides O(1) add, remove, and contains operations.

Why Use It?

When to use HashSet:
- Need to ensure uniqueness
- Fast membership testing (contains)
- Order doesn't matter
- No duplicate values allowed

Interview Answer Template

Interview Answer:
"I used ArrayList for book lists, search results, and catalog displays because it provides O(1) random access by index. This is crucial for pagination - when a user requests page 2 with 12 books per page, I can use subList(12, 24) which is a constant-time operation with ArrayList. It also supports efficient sorting with the sort() method, and getting the total count with size() is O(1). These operations are much slower with LinkedList where random access requires traversing the list."
Concrete example:
"For the catalog page with pagination, I use ArrayList.subList() to get a slice of books for each page. With 100 books and 12 per page, getting page 5 (books 48-59) is instant with ArrayList - just calculate the indices and return the sublist. With LinkedList, I'd have to iterate through 48 elements to reach the starting point, which is wasteful."

General Example

How HashSet works:

```
// HashSet internally uses HashMap
class HashSet<E> {
  private HashMap<E, Object> map;
  private static final Object PRESENT = new Object();

  public boolean add(E element) {
    return map.put(element, PRESENT) == null;
  }

  public boolean contains(E element) {
    return map.containsKey(element);
  }

  public boolean remove(E element) {
    return map.remove(element) == PRESENT;
  }
}

// Usage
Set<String> genres = new HashSet<>();
genres.add("Fantasy");     // true (added)
genres.add("Fiction");     // true (added)
genres.add("Fantasy");     // false (duplicate!)

genres.contains("Fantasy"); // true - O(1)
genres.size();              // 2 (no duplicates)
```

Time Complexity:
- **add:** O(1) average
- **contains:** O(1) average
- **remove:** O(1) average
- **size:** O(1)

In Your Bookstore Project

In Your Bookstore - Unique Genres:

```
// Get all unique genres
public Set<String> getAllGenres() {
  List<Book> allBooks = bookService.listAllBooks();

  // Extract genres into HashSet (removes duplicates)
  return allBooks.stream()
    .map(Book::getGenre)
    .filter(Objects::nonNull)
    .collect(Collectors.toSet());  // Returns HashSet
}

// Example output:
// ["Fantasy", "Fiction", "Mystery", "Programming"]
// Each genre appears only once

// Filter dropdown in frontend
@GetMapping("/genres")
public ResponseEntity<Set<String>> getGenres() {
  Set<String> genres = bookService.getAllGenres();
  return ResponseEntity.ok(genres);
}

// Check if genre exists
public boolean genreExists(String genre) {
  Set<String> genres = getAllGenres();
  return genres.contains(genre);  // O(1)
}

// Get unique authors
public Set<String> getAllAuthors() {
  return bookService.listAllBooks().stream()
    .map(Book::getAuthor)
    .filter(Objects::nonNull)
    .collect(Collectors.toSet());
}

// Remove duplicate book IDs
public List<Book> getUniqueRecommendations(
    String username) {

  List<String> history = historyService.getHistory(username);
  List<String> recommendations =
    recommendationService.getRecommendations(username);

  // Use HashSet to remove books already in history
  Set<String> historySet = new HashSet<>(history);

  return recommendations.stream()
    .filter(bookId -> !historySet.contains(bookId))  // O(1)
    .map(bookService::getBookById)
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
}
```

Why HashSet for genres:
1. **Uniqueness:** Automatically removes duplicate genres
2. **Fast lookup:** Check if genre exists - O(1)
3. **Simple API:** No need to manually check duplicates
4. **Stream integration:** Collectors.toSet() is convenient

HashSet vs ArrayList for genres:

```
// With ArrayList (need manual deduplication)
List<String> genres = new ArrayList<>();
for (Book book : books) {
  String genre = book.getGenre();
  if (!genres.contains(genre)) {  // O(n) check!
    genres.add(genre);
  }
}
// Slow for large lists

// With HashSet (automatic deduplication)
Set<String> genres = new HashSet<>();
for (Book book : books) {
  genres.add(book.getGenre());  // O(1) add
  // Duplicates automatically ignored
}
// Fast regardless of size

// Or with streams
Set<String> genres = books.stream()
  .map(Book::getGenre)
  .collect(Collectors.toSet());
// One line!
```

Interview Answer Template

Interview Answer:
"I used HashSet for collecting unique genres and authors. When displaying a genre filter dropdown, I need each genre to appear only once. Using HashSet, I simply stream through all books, map to their genres, and collect to a Set - duplicates are automatically removed. The contains() operation is O(1), so checking if a genre exists is instant. This is much more efficient than using ArrayList where I'd need to manually check for duplicates with contains() which is O(n) for each check."

Example:
"For recommendation filtering, I convert the user's browsing history to a HashSet. Then when filtering recommended books, I check historySet.contains(bookId) which is O(1). If I used ArrayList for history, this check would be O(n), and checking for each of 100 recommendations would be O(n Œ m) = O(100 Œ 10) = 1000 operations. With HashSet, it's just O(m) = O(100) operations - 10x faster."

ALGORITHMS COMPLEXITY

Big O Notation Explained

Definition

What is Big O?
Big O notation describes the worst-case time or space complexity of an algorithm as input size grows. It ignores constants and focuses on dominant terms.

Common Complexities (fastest to slowest):

- **O(1):** Constant - same time regardless of input
- **O(log n):** Logarithmic - halves each step (binary search)
- **O(n):** Linear - proportional to input size
- **O(n log n):** Linearithmic - efficient sorting
- **O(nš):** Quadratic - nested loops
- **O(2):** Exponential - recursive problems

General Example

Visual Examples:

```
// O(1) - Constant
public Book getById(String id) {
  return map.get(id);  // Always same time
}

// O(log n) - Logarithmic
public int binarySearch(int[] arr, int target) {
  int left = 0, right = arr.length - 1;
  while (left <= right) {
    int mid = (left + right) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
// Each iteration halves search space

// O(n) - Linear
public List<Book> search(String query) {
  List<Book> results = new ArrayList<>();
  for (Book book : allBooks) {  // Loop all books
    if (book.getTitle().contains(query)) {
      results.add(book);
    }
  }
  return results;
}
// Time grows linearly with number of books

// O(n log n) - Linearithmic
public void sortBooks(List<Book> books) {
  books.sort(Comparator.comparing(Book::getPrice));
  // Java uses Tim Sort - O(n log n)
}

// O(nš) - Quadratic
public List<String[]> findDuplicates(List<Book> books) {
  List<String[]> dupes = new ArrayList<>();
  for (int i = 0; i < books.size(); i++) {
    for (int j = i + 1; j < books.size(); j++) {
      if (books.get(i).getTitle()
          .equals(books.get(j).getTitle())) {
        dupes.add(new String[]{
          books.get(i).getTitle(),
          books.get(j).getTitle()
        });
      }
    }
  }
  return dupes;
}
// Nested loops - avoid if possible!
```

Algorithms in Your Project

1. Search Algorithm

```
// Linear search - O(n)
public List<Book> searchBooks(String query) {
  List<Book> allBooks = getAllBooks();  // O(1) if cached
  String lowerQuery = query.toLowerCase();  // O(m) where m = query length

  return allBooks.stream()
    .filter(b ->   // O(n) - check each book
      b.getTitle().toLowerCase().contains(lowerQuery) ||  // O(t) where t = title length
      b.getAuthor().toLowerCase().contains(lowerQuery))   // O(a) where a = author length
    .collect(Collectors.toList());
}

// Total: O(n Œ (t + a))
// For 100 books with avg 20-char titles: ~2000 operations
// Fast enough for small datasets
```

Interview Explanation:
"My search algorithm is O(n) where n is the number of books. It scans through all books and checks if the query appears in the title or author. For each book, the contains() check is O(m) where m is the string length, but since titles and authors are short (usually < 50 characters), this is effectively constant. For 100 books, this completes in under 200ms, which is acceptable for the current scale."

Follow-up: "How would you improve it?":
"For better performance with thousands of books, I'd use Elasticsearch for full-text search. Elasticsearch builds inverted indexes, allowing O(log n) search time. It also supports fuzzy matching (typo tolerance), relevance scoring, and filters. The tradeoff is added complexity and infrastructure. For the current demo with < 1000 books, linear search is sufficient."

2. Sorting Algorithm

```
// Java's sort() uses TimSort - O(n log n)
public List<Book> sortByPrice(List<Book> books) {
  books.sort(Comparator.comparing(Book::getPrice));
  return books;
}

// Time complexity: O(n log n)
// Space complexity: O(n) for merge sort workspace

// For 100 books:
// n log n = 100 Œ log(100) = 100 Œ 6.64  664 comparisons
// For 1000 books:
// n log n = 1000 Œ log(1000) = 1000 Œ 9.97  9970 comparisons
// Scales well!
```

Interview Explanation:
"For sorting books, I use Java's built-in sort() method with Comparator.comparing(). This uses TimSort, a hybrid algorithm combining merge sort and insertion sort with O(n log n) time complexity. For 100 books, this performs about 664 comparisons. I can sort by price, title, author, or stock quantity by passing different comparators. The algorithm is in-place (sorts the original list), which is memory-efficient."

3. ASIN Generation (Hashing)

```
// SHA-256 hashing - O(m) where m = input length
public static String generateFromBook(
    String title, String author) {

  String input = (title + "|" + author)
    .toLowerCase().trim();  // O(m)

  byte[] hash = sha256(input);  // O(m)

  StringBuilder sb = new StringBuilder("B0");
  for (int i = 0; i < 8; i++) {  // O(1) - fixed 8 iterations
    int index = Math.abs(hash[i]) % 36;
    sb.append(CHARS.charAt(index));
  }

  return sb.toString();
}

// Total: O(m) where m = title + author length
// For "Clean Code|Robert Martin" (24 chars): ~24 operations
// Very fast!
```

Interview Explanation:
"ASIN generation uses SHA-256 cryptographic hashing, which is O(m) where m is the input length (title + author). The hash function processes each character once, producing a fixed 256-bit output. I then take the first 8 bytes and convert them to alphanumeric characters. Since titles and authors are typically short (< 100 characters), this completes in microseconds. The deterministic nature means the same book always gets the same ASIN, preventing duplicates."

4. Cart Total Calculation

```java
// Sum calculation - O(n) where n = cart items
public BigDecimal calculateTotal() {
  return items.values().stream()
    .map(item ->
      item.getUnitPrice()
        .multiply(BigDecimal.valueOf(item.getQuantity())))
    .reduce(BigDecimal.ZERO, BigDecimal::add);
}

// For cart with 5 items: 5 multiplications + 5 additions
// Time: O(n) but n is small (usually < 20 items)
// Space: O(1) - no extra collections
```

Interview Explanation:
"Calculating cart total is O(n) where n is the number of items in the cart. For each item, I multiply unit price by quantity and sum them up. Since carts typically have < 20 items, this is very fast. I use BigDecimal for exact decimal arithmetic to avoid rounding errors with money. The reduce() operation accumulates the sum without creating intermediate collections, making it space-efficient."

5. Pagination

```java
// Pagination - O(1) for sublist
public List<Book> getPage(
    List<Book> books, int page, int size) {

  int start = page * size;  // O(1)
  int end = Math.min(start + size, books.size());  // O(1)

  return books.subList(start, end);  // O(1) with ArrayList
  // Returns view, doesn't copy elements
}

// Total: O(1)
// Same time for page 1 or page 100!
```

Interview Explanation:
"Pagination with ArrayList is O(1) because subList() returns a view of the original list, not a copy. It just stores start and end indices. Whether requesting page 1 or page 100, the time is constant. This is why I chose ArrayList over LinkedList for book lists - LinkedList.subList() would be O(n) since it needs to traverse to the starting index."

TESTING STRATEGIES

Unit Testing with JUnit

Definition

What is Unit Testing?
Unit testing tests individual components (methods, classes) in isolation. Mock dependencies to focus on one unit of code. Fast execution, easy to debug.

General Example

Simple Unit Test:

```java
@Test
public void testAddition() {
  // Arrange
  Calculator calc = new Calculator();

  // Act
  int result = calc.add(2, 3);

  // Assert
  assertEquals(5, result);
}

@Test
public void testDivisionByZero() {
  Calculator calc = new Calculator();

  // Assert exception thrown
  assertThrows(ArithmeticException.class, () -> {
    calc.divide(10, 0);
  });
}
```

In Your Bookstore Project

Your Unit Tests:

```java
// CartTest.java
@Test
public void testAddBook() {
  // Arrange
  Cart cart = new Cart();
  Book book = new Book();
  book.setId("b1");
  book.setTitle("Test Book");
  book.setPrice(new BigDecimal("19.99"));

  // Act
  cart.addBook(book, 2);

  // Assert
  assertEquals(1, cart.size());
  assertEquals(new BigDecimal("39.98"),
               cart.calculateTotal());
}

@Test
public void testAddSameBookTwice() {
  Cart cart = new Cart();
  Book book = createTestBook("b1", "Book", "19.99");

  cart.addBook(book, 2);
  cart.addBook(book, 3);  // Add again

  // Should update quantity, not add new item
  assertEquals(1, cart.size());
  CartItem item = cart.getItems().get("b1");
  assertEquals(5, item.getQuantity());
}

@Test
public void testRemoveBook() {
  Cart cart = new Cart();
  Book book = createTestBook("b1", "Book", "19.99");
  cart.addBook(book, 2);

  cart.removeBook("b1");

  assertTrue(cart.isEmpty());
  assertEquals(BigDecimal.ZERO, cart.calculateTotal());
}

// AsinGeneratorTest.java
@Test
public void testGenerateFromBookIsDeterministic() {
  String asin1 = AsinGenerator.generateFromBook(
    "Clean Code", "Robert Martin");
  String asin2 = AsinGenerator.generateFromBook(
    "Clean Code", "Robert Martin");

  // Same inputs  same output
  assertEquals(asin1, asin2);
}

@Test
public void testGeneratedAsinIsValid() {
  String asin = AsinGenerator.generateFromBook(
    "Test", "Author");

  assertTrue(AsinGenerator.isValid(asin));
  assertTrue(asin.startsWith("B0"));
  assertEquals(10, asin.length());
}

@Test
public void testDifferentBooksGetDifferentAsins() {
  String asin1 = AsinGenerator.generateFromBook(
    "Book A", "Author 1");
  String asin2 = AsinGenerator.generateFromBook(
    "Book B", "Author 2");

  assertNotEquals(asin1, asin2);
}
```

Mocking with Mockito

Definition

What is Mocking?
Mocking creates fake objects that simulate real dependencies. Allows testing in isolation without databases, external APIs, or other services.

General Example

Simple Mock Example:

```
@Test
public void testUserService() {
  // Create mock repository
  UserRepository mockRepo = mock(UserRepository.class);

  // Define mock behavior
  User fakeUser = new User("john", "john@example.com");
  when(mockRepo.findByUsername("john"))
    .thenReturn(fakeUser);

  // Inject mock into service
  UserService service = new UserService(mockRepo);

  // Test service logic
  User result = service.getUser("john");
  assertEquals("john", result.getUsername());

  // Verify repository was called
  verify(mockRepo).findByUsername("john");
}
```

In Your Bookstore Project

Your Mock Tests:

```
@ExtendWith(MockitoExtension.class)
public class CheckoutServiceTest {

  @Mock
  private BookService bookService;

  @Mock
  private OrderRepository orderRepository;

  @InjectMocks
  private CheckoutService checkoutService;

  @Test
  public void testSuccessfulCheckout() {
    // Arrange
    Cart cart = new Cart();
    Book book = createTestBook("b1", "Book", "19.99");
    cart.addBook(book, 2);

    // Mock book service to return book with stock
    when(bookService.getBookById("b1"))
      .thenReturn(book);
    book.setStockQuantity(10);  // Enough stock

    // Act
    Order order = checkoutService.checkout(cart, "user1");

    // Assert
    assertNotNull(order.getOrderId());
    assertEquals("user1", order.getUsername());
    assertEquals(new BigDecimal("39.98"), order.getTotal());

    // Verify interactions
    verify(bookService).getBookById("b1");
    verify(orderRepository).save(any(Order.class));
  }

  @Test
  public void testCheckoutWithInsufficientStock() {
    Cart cart = new Cart();
    Book book = createTestBook("b1", "Book", "19.99");
    cart.addBook(book, 5);  // Want 5 books

    book.setStockQuantity(3);  // Only 3 in stock
    when(bookService.getBookById("b1"))
      .thenReturn(book);

    // Should throw exception
    assertThrows(InsufficientStockException.class, () -> {
      checkoutService.checkout(cart, "user1");
    });

    // Order should NOT be saved
    verify(orderRepository, never()).save(any());
  }

  @Test
  public void testCheckoutWithEmptyCart() {
    Cart cart = new Cart();  // Empty

    assertThrows(EmptyCartException.class, () -> {
      checkoutService.checkout(cart, "user1");
    });

    // No interactions with dependencies
    verifyNoInteractions(bookService);
    verifyNoInteractions(orderRepository);
  }
}
```

Interview Answer Template

Interview Answer:
"I wrote 134 unit tests achieving 58% code coverage. I used JUnit 5 for test framework and Mockito for mocking dependencies. For example, when testing CheckoutService, I mocked BookService and OrderRepository so the test doesn't need a real database. I define mock behavior with when().thenReturn() and verify interactions with verify(). This lets me test edge cases like insufficient stock or empty carts without complex setup."
Follow-up: "Why mock dependencies?":
"Mocking isolates the unit under test. If CheckoutService tests fail, I know the bug is in CheckoutService, not in BookService or database code. Tests run in milliseconds without starting DynamoDB. I can simulate error conditions like database failures that are hard to reproduce in integration tests. This makes tests fast, reliable, and focused."

QUICK REFERENCE

Project Stats (Memorize!)

- **Duration:** 10 days
- **Backend:** 42 Java classes, 3,500 LOC
- **Frontend:** 25 React components, 2,800 LOC
- **Total Code:** 6,300+ lines
- **API Endpoints :** 20+ RESTful endpoints
- **Tests:** 134 tests, 58% coverage
- **Features:** 11 core + 3 advanced
- **Tech Stack:** Java 17, Spring Boot 3, React 18, DynamoDB, Material-UI
- **Git:** 100+ commits
- **CI/CD:** GitHub Actions (3 min build)

Time Complexities Cheat Sheet

Data Structure Operations:

- **HashMap:** get O(1), put O(1), remove O(1)
- **LinkedList:** addFirst O(1), removeLast O(1), get(i) O(n)
- **ArrayList:** get(i) O(1), add O(1), add(0) O(n)
- **HashSet:** add O(1), contains O(1), remove O(1)

Algorithms in Project:

- **Book Search:** O(n) - linear scan
- **Sort Books:** O(n log n) - TimSort
- **ASIN Generation:** O(m) - SHA-256, m = input length
- **Cart Total:** O(n) - n = cart items
- **Pagination:** O(1) - ArrayList.subList()
- **History Add:** O(1) - LinkedList.addFirst()

Design Patterns Summary

1. **Repository:** Separates data access (BookService)
2. **Adapter:** BookServiceAdapter bridges layers
3. **Factory:** AsinGenerator, DeterministicId
4. **Singleton:** Spring @Service beans (one instance)
5. **Strategy:** Sorting comparators (price/title/author)
6. **Builder:** DynamoDB SDK (Key.builder())

SOLID Principles Summary

1. **Single Responsibility:** Each class has one job
2. **Open/Closed:** Extend via interfaces, don't modify
3. **Liskov Substitution:** Child classes are interchangeable
4. **Interface Segregation:** Small focused interfaces
5. **Dependency Inversion:** Depend on abstractions

Key Technical Decisions

1. **ASIN System:** Amazon-style IDs (B0XXXXXXXX)
2. **Deterministic IDs:** SHA-1 hash prevents duplicates
3. **LinkedHashMap for Cart:** Preserves insertion order
4. **LinkedList for History:** Efficient FIFO with capacity
5. **BigDecimal for Money:** Exact decimal arithmetic
6. **Optional for Nulls:** Type-safe null handling
7. **Synchronized Cart:** Thread-safe for concurrent requests
8. **Repository Pattern:** Abstracts database operations

BEHAVIORAL INTERVIEW ANSWERS

STAR Method Template

STAR Framework:

- **S**ituation: Context and background (1-2 sentences)
- **T**ask: Your responsibility or goal (1 sentence)
- **A**ction: What YOU did (3-4 sentences, use "I")
- **R**esult: Outcome with metrics (1-2 sentences)

Question: Tell me about a challenging bug

Interview Answer Template

Situation: During bulk book imports, the system was creating duplicate entries for the same book. I'd upload a CSV with 50 books, and if I ran it twice, I'd have 100 books instead of 50 updated books.
Task: I needed to ensure the same book always gets the same ID so re-importing would update existing books instead of creating duplicates.
Action: I investigated and found the issue was random UUID generation - each import created new IDs. I researched solutions and decided to implement deterministic ID generation using SHA-1 hashing of (title + author). I created a DeterministicId utility class that generates the same ID for the same book. I also refactored the save method to check if a book with the same title exists and update it instead of inserting. I wrote unit tests to verify the same book always gets the same ID, and integration tests to confirm bulk imports are idempotent.
Result: Completely eliminated duplicates - 100% success rate. Bulk import became idempotent, meaning admins could safely re-run imports to update prices or stock without worrying about duplicates. This saved hours of manual cleanup.

Question: Describe a time you had to learn something quickly

Interview Answer Template

Situation: I had 10 days to build a full-stack bookstore application but had never used Spring Boot, React, or DynamoDB before. I only had basic Java knowledge from coursework.
Task: I needed to learn three new technologies simultaneously and deliver a working application with all required features.
Action: I broke down the learning into phases. Days 1-2: I followed Spring Boot tutorials and built a simple REST API. Days 3-4: I learned React basics and created components. Days 5-6: I integrated DynamoDB for user management and studied NoSQL patterns. Throughout, I used official documentation as my primary resource and AI tools (Claude) to explain concepts I didn't understand. When stuck, I'd search Stack Overflow for similar problems. I prioritized learning what I needed for each feature rather than trying to learn everything upfront.
Result: Successfully delivered all 11 core features plus 3 advanced features on time. The application has 134 passing tests and 58% code coverage. More importantly, I can now confidently work with Spring Boot, React, and DynamoDB - skills I'll use at Amazon. The key was structured learning and not being afraid to ask for help.

Question: Tell me about a time you disagreed with a decision

Interview Answer Template

Situation: While working with Claude AI assistant, it suggested using MongoDB for my bookstore database. However, the project requirements specified using AWS technologies.
Task: I needed to evaluate the suggestion and make an informed decision about which database to use.
Action: I researched both options thoroughly. MongoDB pros: easier local setup, familiar SQL-like queries. DynamoDB pros: AWS native, serverless scaling, no server management, aligns with Amazon's tech stack. I considered the project context - this was for Amazon's ATLAS program, so demonstrating AWS skills was important. I also thought about production scenarios where DynamoDB's auto-scaling would be valuable. I respectfully explained my reasoning to the AI and decided to use DynamoDB despite the steeper learning curve.
Result: Made the right choice for the context. I learned DynamoDB's partition key design, GSIs, and batch operations - skills directly applicable to Amazon. The project demonstrates AWS expertise, which strengthened my candidacy. This taught me to evaluate suggestions critically rather than blindly following advice, even from experts.

Question: Describe a time you made a mistake

Interview Answer Template

Situation: Early in the project, I spent an entire day trying to optimize DynamoDB queries before confirming the basic search functionality even worked correctly.
Task: I was supposed to implement book search, but I got sidetracked by performance optimization.
Action: I realized I was doing premature optimization - a common anti-pattern. I stopped, took a step back, and reassessed. I implemented the simplest version first (linear search through all books) and verified it worked correctly with tests. Only after confirming correctness did I measure performance and found it was already fast enough for the current scale (< 200ms for 100 books). I documented this experience and now follow "make it work, make it right, make it fast" principle.
Result: Saved 6+ hours that I redirected to implementing bulk operations and CSV import features. The lesson was valuable: validate your approach works before optimizing. Now I always start with the simplest solution, test it, measure performance, and optimize only if needed. This made me more productive and pragmatic.

Question: Tell me about a time you had to meet a tight deadline

Interview Answer Template

Situation: I had 10 days to build the entire bookstore application from scratch, including learning new technologies. By day 7, I still needed to implement testing, CI/CD, and polish the UI.
Task: Complete all remaining features, achieve good test coverage, and prepare for demo presentation in 3 days.
Action: I prioritized ruthlessly. I listed all remaining tasks and categorized them: must-have (tests, CI/CD, core features), nice-to-have (animations, advanced filters), and can-skip (email notifications, advanced analytics). I focused on must-haves first. I worked in focused 2-hour blocks with short breaks to maintain quality. For CI/CD, I used GitHub Actions templates rather than building from scratch. For tests, I focused on critical paths (checkout, cart, ASIN generation) rather than 100% coverage. I cut scope on less important features like social sharing.
Result: Delivered on time with 134 tests (58% coverage), working CI/CD pipeline, and polished demo. All core requirements met plus 3 advanced features. The key was distinguishing between essential and nice-to-have, and being willing to cut scope intelligently. The demo was successful, and I had time to prepare a comprehensive presentation.

Question: How do you handle ambiguity?

Interview Answer Template

Situation: The project requirements said to implement "recommendations" but didn't specify the algorithm, data source, or how personalized they should be.
Task: I needed to design and implement a recommendation system with minimal guidance.
Action: I broke down the ambiguity into specific questions: What data do I have? (browsing history). What's the simplest approach? (recommend books in same genre). What's feasible in the timeline? (avoid complex ML models). I researched common recommendation approaches: collaborative filtering (needs many users), content-based (uses book attributes), and popularity-based (fallback). I decided on a hybrid approach: content-based using browsing history genres, with popularity fallback for new users. I started with the simplest version that works, tested it, and documented assumptions. I made sure to build in flexibility to swap algorithms later.
Result: Delivered working recommendations that make sense to users (books in genres they've browsed). The system is extensible - I can add collaborative filtering later without changing the interface. This taught me to start simple, make assumptions explicit, and build for flexibility when requirements are unclear.

Ownership

Interview Answer Template

Example from project:
"When I found duplicate books appearing after imports, I could have just documented it as a 'known issue' and moved on. Instead, I took ownership. I spent 4 hours researching solutions, implementing deterministic IDs, and writing tests to prevent regression. Even though it wasn't blocking the demo, I knew it would cause problems for real users, so I fixed it properly. I also added documentation explaining the ID generation logic for future developers."
Another example:
"I set up CI/CD even though it wasn't required because I wanted to ensure code quality. When the pipeline failed due to DynamoDB not being available, I could have removed it. Instead, I researched how to run DynamoDB Local in Docker and configured the pipeline properly. I owned the problem end-to-end."

Question: Describe a time you took initiative

Interview Answer Template

Situation: The project requirements didn't specify needing a CI/CD pipeline - just that code should be tested. Many students were submitting projects without any automated testing.
Task: I wanted to demonstrate professional software engineering practices beyond the minimum requirements.
Action: I researched GitHub Actions and found it integrates well with GitHub repos. I spent 3 hours on day 9 setting up a CI/CD pipeline that runs tests automatically on every commit and pull request. I configured it to run both backend (Maven) and frontend (npm) builds in parallel. I added status badges to the README to show build status. I also configured the pipeline to fail if tests don't pass, enforcing quality gates. When I encountered issues with DynamoDB not being available in CI, I researched and added DynamoDB Local as a Docker service.
Result: Every commit is now automatically tested, catching bugs before they reach the demo. The pipeline has already caught 3 bugs I would have missed in manual testing. This shows initiative and demonstrates I understand modern DevOps practices. It also made me more confident in my code quality - if the build is green, I know tests pass.

AMAZON LEADERSHIP PRINCIPLES

Customer Obsession

Interview Answer Template

Example from project:
"When designing the browsing history feature, I limited it to 10 books because research shows users get overwhelmed with long lists. I also added book covers from OpenLibrary API because visual recognition is faster than reading titles. When covers failed to load, I created gradient fallbacks with book titles so users never see broken images. Every decision prioritized user experience over technical convenience."
Follow-up: "How did you validate this was good for customers?":
"I manually tested the user flow: browse catalog view book add to cart checkout. I timed how long each step took and looked for friction points. I also showed the app to 3 friends and watched them use it without instruction. Their feedback led to improvements like larger 'Add to Cart' buttons and clearer cart totals. This taught me to observe actual users, not just assume what they want."

Invent and Simplify

Interview Answer Template

Example from project:
"For bulk operations, I could have built separate endpoints for bulk-create, bulk-update, bulk-delete. Instead, I invented a simpler approach: one bulk endpoint that accepts a list of books and automatically determines whether to create or update based on whether the book exists. This reduced API complexity from 3 endpoints to 1, making it easier for admins to use."
ASIN System:
"I invented a deterministic ASIN generation system inspired by Amazon but simplified for books. Amazon's ASINs are random for general products, but for books, I generate them deterministically from title+author. This prevents duplicates elegantly without needing a separate deduplication system. Simple solution to a complex problem."

Learn and Be Curious

Interview Answer Template

Example from project:
"I had never used Spring Boot, React, or DynamoDB before this project. Rather than just copying tutorials, I dove deep into documentation to understand WHY things work. For example, I learned about DynamoDB's partition key design principles, not just how to write queries. This deeper understanding helped me design a better schema with proper GSIs. I also learned about Java Streams, Optional, and BigDecimal - modern Java features I hadn't encountered in coursework."
Continuous learning:
"When implementing the cart, I learned about thread safety and the 'synchronized' keyword. This led me to research Java concurrency, which helped me understand why Spring services are singleton by default and how Tomcat handles multiple requests. I always ask 'why' and research beyond the immediate problem."

Insist on the Highest Standards

Interview Answer Template

Example from project:
"I could have shipped with 20% test coverage, but I insisted on achieving 58% with 134 tests. I wrote tests for edge cases like empty carts, insufficient stock, and invalid input. I set up CI/CD to enforce these standards - code can't be merged if tests fail. I also used BigDecimal for all money calculations instead of doubles, even though doubles would be easier, because financial calculations must be exact."
Code quality:
"I followed SOLID principles and design patterns even though they took more time initially. I refactored code when I saw duplication or tight coupling. For example, I created BookServiceAdapter to clean up controller code even though controllers worked without it. The result is maintainable, professional-grade code."

Bias for Action

Interview Answer Template

Example from project:
"When I encountered the empty string ID bug in DynamoDB, I could have spent hours researching the perfect solution. Instead, I quickly implemented a fix (generate ID if missing or empty) and moved on. I shipped the fix in 30 minutes and validated it worked. Later, I could always optimize if needed, but bias for action meant users weren't blocked."
Experimentation:
"I experimented with different UI libraries (Material-UI vs Ant Design) by building small prototypes. After 2 hours, I chose Material-UI and committed. I didn't let perfect be the enemy of good - I made a decision with limited information and moved forward. This velocity was crucial for meeting the 10-day deadline."

Hire and Develop the Best

Interview Answer Template

Example from project:
"While this was a solo project, I wrote comprehensive documentation and clean code thinking about future developers who might work on this. I added JavaDoc comments explaining non-obvious logic, especially for ASIN generation and deterministic IDs. I structured the code with clear separation of concerns so a new developer could understand the flow. I also wrote a detailed README with setup instructions, architecture diagrams, and API documentation."
Mentoring mindset:
"I documented every major decision in commit messages - not just 'fixed bug' but 'Fixed duplicate books by implementing deterministic ID generation using SHA-1 hash of title+author'. This teaches future developers (or future me) the reasoning behind decisions."

Think Big

Interview Answer Template

Example from project:
"While building for 100 books, I designed the architecture to scale to thousands. I chose DynamoDB because it scales horizontally. I implemented pagination early, even though it wasn't needed for 100 books, because I knew it would be essential at scale. I designed the ASIN system to handle billions of books ($36^8 = 2.8 trillion possible ASINs$). $This forward-thinking approach means the system can grow without major rewrites$
Future vision:
"I documented future enhancements like Elasticsearch for search, Redis for caching, and machine learning for recommendations. While I didn't implement them in 10 days, I designed the architecture so they can be added later without breaking existing code. This shows I'm thinking beyond the immediate requirements."

Frugality

Interview Answer Template

Example from project:
"Instead of paying for image hosting, I integrated with OpenLibrary's free API for book covers. Instead of using paid services like Auth0, I implemented session-based authentication myself. Instead of deploying to AWS (which costs money), I documented local setup instructions. I used free tools: GitHub for version control, GitHub Actions for CI/CD (free for public repos), and open-source libraries. The entire project cost $0 to build and run."
Resource efficiency:
"I reused Spring Boot's built-in features instead of adding dependencies. I used Java's standard library (BigDecimal, Optional, Streams) instead of external libraries. This kept the application lightweight and reduced potential security vulnerabilities from third-party dependencies."

Deliver Results

Interview Answer Template

Example from project:
"I had 10 days and delivered all 11 core requirements plus 3 advanced features. The application has 134 passing tests, a working CI/CD pipeline, and comprehensive documentation. I met every deadline I set for myself. More importantly, the application works reliably - zero crashes during demo, all features functional, good performance. I didn't just deliver code; I delivered a working product."

Metrics:
"Concrete results: 6,300+ lines of code, 58% test coverage, 20+ API endpoints, 100+ Git commits, sub-200ms response times, zero production bugs. These aren't just numbers - they represent a fully functional e-commerce platform built in 10 days."

COMMON PITFALLS TO AVOID

During Technical Questions

DON'T:
- Say "I don't know" without trying to reason through it
- Jump into coding without clarifying requirements
- Ignore edge cases (null inputs, empty lists, etc.)
- Write code silently - always explain your thinking
- Assume the interviewer understands your code
- Give up when stuck - ask for hints
- Forget to test your code with examples

DO:
- Ask clarifying questions before starting
- Explain your approach before coding
- Consider edge cases and mention them
- Think aloud - show your problem-solving process
- Walk through your code with examples
- Admit when you're unsure and reason through it
- Discuss time/space complexity

During Behavioral Questions

DON'T:
- Ramble without structure (use STAR)
- Speak in generalities ("we did this")
- Blame others for failures
- Give answers over 3 minutes
- Make up stories - be authentic
- Focus only on positive outcomes - show learning from failures

DO:
- Use STAR method (Situation, Task, Action, Result)
- Use "I" not "we" - show YOUR contribution
- Take ownership of mistakes
- Include metrics and specific outcomes
- Be honest about challenges faced
- Show what you learned from experiences

Common Mistakes About Your Project

Avoid these misconceptions:

1. **WRONG:** "I used AI so I didn't really code it"
   RIGHT: "I used AI as a learning tool, but I understand every line of code and made all design decisions"
2. **WRONG:** "It's just a simple CRUD app"
   RIGHT: "It's a full-stack e-commerce platform with advanced features like deterministic ID generation, bulk operations, and real-time recommendations"
3. **WRONG:** "I don't know why I used HashMap, Claude told me to"
   RIGHT: "I used HashMap for O(1) lookup, which is crucial for cart operations where users frequently check if items exist"
4. **WRONG:** "I just followed tutorials"
   RIGHT: "I learned from documentation and applied concepts to solve my specific problems, like designing the ASIN system"
5. **WRONG:** "The tests don't really matter"
   RIGHT: "Tests are crucial - they caught 3 bugs during development and give me confidence to refactor code"

INTERVIEW TIPS TRICKS

Before the Interview

24 Hours Before:
1. Review this cheatsheet (focus on STAR answers)
2. Run your application locally - ensure it works
3. Practice explaining your project in 2 minutes
4. Prepare 3 questions to ask interviewer
5. Test your video/audio setup
6. Have project report and GitHub repo open
7. Get good sleep (8 hours)

1 Hour Before:
1. Review key stats (6,300 LOC, 134 tests, 58% coverage)
2. Practice STAR answer for "challenging bug" (duplicates)
3. Test internet connection and video call software
4. Have water nearby (stay hydrated)
5. Close all distracting apps/tabs
6. Do 5 deep breaths (calm nerves)

During the Interview

First 5 Minutes:
- Smile and be enthusiastic
- Listen carefully to introduction
- Take notes on paper (not laptop - shows engagement)
- Ask how they'd like to structure the interview

Technical Questions:
1. **Clarify:** "Can I ask a few questions about the requirements?"
2. **Plan:** "Here's my approach..." (outline before coding)
3. **Code:** Explain as you write, not after
4. **Test:** Walk through with example inputs
5. **Optimize:** Discuss time/space complexity
6. **Edge cases:** "What if the input is null/empty?"

Behavioral Questions:
1. **Pause:** Take 3-5 seconds to think
2. **Structure:** "Let me use the STAR method..."
3. **Be concise:** 2-3 minutes max
4. **Check:** "Does that answer your question?"
5. **Connect:** "This relates to [Leadership Principle]"

When Stuck:
- "I'm not immediately sure, but let me think through it..."
- "Can you give me a hint about..."
- "Here's what I know... [reason through it]"
- "In my project, I solved something similar by..."

Body Language Communication

- **Eye contact:** Look at camera, not your screen
- **Posture:** Sit up straight, lean slightly forward
- **Hands:** Use gestures naturally, don't fidget
- **Pace:** Speak clearly, not too fast
- **Energy:** Show enthusiasm for technology
- **Smile:** Especially when talking about your project

Questions to Ask Interviewer

Good questions (shows curiosity):

1. "What's the tech stack for the team I'd be joining?"
2. "What does a typical day look like for an SDE intern?"
3. "What kind of projects do interns typically work on?"
4. "How does Amazon support intern learning and development?"
5. "What's your favorite part about working at Amazon?"
6. "How do teams practice Amazon's Leadership Principles daily?"

Avoid:

- Questions about salary/benefits (save for recruiter)
- Questions easily answered by Google
- "How did I do?" (wait for feedback)
- Negative questions about work-life balance

</> CODE SNIPPETS TO MEMORIZE

ASIN Generation (Critical to Explain)

```
// Deterministic ASIN from book
public static String generateFromBook(
    String title, String author) {

  // 1. Create input string
  String input = (title + "|" + author)
    .toLowerCase().trim();

  // 2. Hash with SHA-256
  byte[] hash = sha256(input);

  // 3. Convert to alphanumeric
  StringBuilder asin = new StringBuilder("B0");
  for (int i = 0; i < 8; i++) {
    int index = Math.abs(hash[i]) % 36;
    asin.append(CHARS.charAt(index));
  }

  return asin.toString();
}

// Same book  same ASIN (prevents duplicates!)
```

Browsing History (LinkedList Usage)

```
public void addBookToHistory(
    String username, String bookId) {

  LinkedList<String> history = getHistory(username);

  // Remove if exists (avoid duplicates)
  history.remove(bookId);  // O(n) but n  10

  // Add to front (most recent)
  history.addFirst(bookId);  // O(1)

  // Maintain capacity
  while (history.size() > MAX_CAPACITY) {
    history.removeLast();  // O(1)
  }
}
```

Cart Total Calculation

```
public BigDecimal calculateTotal() {
  return items.values().stream()
    .map(item ->
      item.getUnitPrice()
        .multiply(BigDecimal.valueOf(
          item.getQuantity())))
    .reduce(BigDecimal.ZERO,
            BigDecimal::add);
}
// Uses BigDecimal for exact money calculations
```

Repository Pattern Structure

```
// Service (Repository)
@Service
public class BookService {
  private final DynamoDbTable<Book> bookTable;

  public List<Book> searchBooks(String query) {
    // Data access logic
  }
}

// Controller
@RestController
public class BooksController {
  private final BookServiceAdapter adapter;

  @GetMapping("/search")
  public List<Book> search(@RequestParam String q) {
    return adapter.search(q);  // No DB logic!
  }
}
```

FINAL CONFIDENCE BOOSTERS

What Makes Your Project Strong

You built a REAL application, not just a tutorial:

1. **Production-ready:** 134 tests, CI/CD, error handling
2. **Innovative:** ASIN system, deterministic IDs (original solutions)
3. **Scalable:** Designed for growth (DynamoDB, pagination, stateless)
4. **Professional:** Follows SOLID, uses design patterns, clean code
5. **Complete:** Frontend + Backend + Database + Tests + Deployment
6. **Documented:** Comprehensive report, API docs, README
7. **Metrics:** Concrete numbers (6,300 LOC, 58% coverage, 20+ APIs)

You Know More Than You Think

Skills you demonstrated:

- **Backend:** Java, Spring Boot, REST APIs, DynamoDB
- **Frontend:** React, Material-UI, responsive design
- **Data Structures:** HashMap, LinkedList, ArrayList, HashSet
- **Algorithms:** Hashing, searching, sorting, pagination
- **Design:** OOP, SOLID, 6 design patterns
- **Testing:** JUnit, Mockito, integration tests
- **DevOps:** Git, GitHub Actions, Docker
- **Problem-solving:** Debugged duplicates, empty IDs, cropping
- **Learning:** Taught yourself 3 technologies in 10 days

You're ready for this interview!

Remember: They Want You to Succeed

Interview mindset:

- Interviewers are evaluating POTENTIAL, not perfection
- It's okay to not know everything - show how you'd figure it out
- They want to see how you think, not just the answer
- Mistakes are learning opportunities - explain what you'd do differently
- You're interviewing them too - assess if it's a good fit
- Be yourself - authenticity is valued at Amazon

You've got this! Trust your preparation.

LAST-MINUTE CHECKLIST

30 Minutes Before Interview

Technical Prep:

- Application running locally (verify it works)
- GitHub repo open in browser
- Project report PDF open
- This cheatsheet visible on second screen
- Code editor ready (in case of coding question)
- Calculator ready (for complexity calculations)

Environment Prep:

- Quiet room, door closed
- Phone on silent
- Video/audio tested
- Good lighting (face visible)
- Professional background (or virtual)
- Water bottle nearby
- Pen and paper for notes

Mental Prep:

- Reviewed elevator pitch (30 seconds)
- Reviewed STAR answer for "challenging bug"
- Reviewed key stats (6,300 LOC, 134 tests, 58%)
- Deep breathing (5 breaths, calm nerves)
- Positive affirmation: "I'm prepared and capable"

Opening Lines (Memorize)

Interview Answer Template

When asked "Tell me about yourself":
"I'm Aditya, part of Amazon's ATLAS program. I recently built a full-stack online bookstore application using Java Spring Boot and React. The project features 20+ REST APIs, DynamoDB integration, and an Amazon-style ASIN system for inventory management. I achieved 58% test coverage with 134 automated tests and set up a CI/CD pipeline. I'm passionate about building scalable systems and learning new technologies quickly. I'm excited about this SDE intern role because..."

When asked "Tell me about your project":
"I built a production-ready e-commerce bookstore in 10 days. It includes all core features - search, cart, checkout, personalized recommendations, and an admin panel. I implemented an innovative ASIN generation system that prevents duplicate books by creating deterministic IDs from title and author. The application has 6,300+ lines of code, integrates with DynamoDB for cloud storage, and uses React with Material-UI for a responsive frontend. Would you like me to dive deeper into any particular aspect?"

Closing Strong

Interview Answer Template

When they ask "Any questions for me?":
"Yes, I have a few questions:

1. What kind of projects do SDE interns typically work on in your team?
2. How does Amazon support interns in learning and growing technically?
3. What's your favorite Amazon Leadership Principle, and how do you see it practiced daily?

Also, I want to express my strong interest in this role. Building this bookstore project reinforced my passion for backend systems and scalable architectures. I'm eager to contribute to Amazon's mission and learn from experienced engineers."

❤ FINAL WORDS

YOU'VE GOT THIS!

You built a full-stack application from scratch in 10 days.
You learned 3 new technologies simultaneously.
You solved complex problems like duplicate prevention.
You wrote 134 tests and set up CI/CD.
You demonstrated all OOP and SOLID principles.

You're more prepared than you realize.

Remember:

- Breathe deeply and stay calm
- Think aloud - show your process
- Use STAR method for behavioral questions
- Ask clarifying questions
- Be authentic and enthusiastic
- They're evaluating potential, not perfection

Good luck with your Amazon SDE Intern interview!

You've prepared well. Trust yourself.

Created by Aditya Shubham
Amazon ATLAS Program
Online Bookstore Application
GitHub: MinotaurG/Online_Bookstore_Application